

AgriXplain: A Full-Stack Internet of Things Smart Farming Platform with Multimodal Sensing, Cloud Analytics, and the AgriFusion Explainable Ensemble Classifier

**Note: Sub-titles are not captured in Xplore and should not be used*

1st Kshitija Sole
Dept. of Computer Science
Institution Name
City, Country
email@institution.edu

4th Author Four
Dept. of AI
Institution Name
City, Country
author4@institution.edu

2nd Author Two
Dept. of Electronics
Institution Name
City, Country
author2@institution.edu

5th Author Five
Dept. of IoT
Institution Name
City, Country
author5@institution.edu

3rd Author Three
Dept. of Agriculture
Institution Name
City, Country
author3@institution.edu

6th Author Six
Dept. of Data Science
Institution Name
City, Country
author6@institution.edu

Abstract—A practical precision agriculture platform must do three things coherently: sense the field with sufficient resolution, translate that data into actionable yield risk predictions, and explain those predictions clearly enough that a farmer with no machine learning background can verify and act on them. Most open-source implementations handle one of these reasonably well and treat the others as out of scope. AgriXplain is our effort to close that gap in a single, auditable, self-hostable stack. We built a multi-sensor edge station around the ESP32-WROOM-32 microcontroller that acquires air temperature and humidity, volumetric soil moisture, soil pH, light intensity, binary rainfall state, water flow rate, wind pulse proxy, and soil nitrogen-phosphorus-potassium via Modbus RTU—ten modalities in total. Telemetry is transmitted as structured JSON over Wi-Fi to a FastAPI backend where four inference models run in parallel on every reading: Random Forest, XGBoost, Bayesian Ridge, and a proposed ensemble we call AgriFusion, which blends the RF and XGBoost class probability vectors with fixed weights (0.35 and 0.65) and applies disagreement-aware confidence damping when the two base learners predict different classes. Per-reading SHAP attributions are computed via TreeSHAP and stored as JSON alongside every database record, making the reasoning behind every risk label inspectable without a data scientist. On the synthetic benchmark dataset shipped with the repository, AgriFusion reaches 70.68% accuracy and 69.97% weighted F1—marginally above both base learners—on a stratified 80/20 hold-out. We are explicit that these numbers reflect benchmark performance on heuristically labeled synthetic data, not

validated agronomic prediction; the benchmark exists to verify that the inference pipeline runs correctly, not to make claims about real-world crops. The web application is implemented in Next.js 14 (App Router) with Tailwind CSS and shaden/ui, packaged for Android via Capacitor 6, and provides role-differentiated farmer and admin dashboards with live sensor cards, SHAP explanation panels, multi-farm analytics, and data export. Hardware calibration constants are explicit in firmware headers, model artifacts are version-tracked files, and the entire stack runs on a single local server with no mandatory cloud subscription.

Keywords—smart agriculture, Internet of Things, precision farming, ESP32, embedded systems, Modbus RTU, soil sensors, cloud computing, machine learning, ensemble methods, Random Forest, XGBoost, explainable artificial intelligence, SHAP, AgriFusion, Next.js, Capacitor, computer vision, plant disease detection, sensor networks, FastAPI

I. INTRODUCTION

Somewhere around 2019, building a Wi-Fi-connected soil sensor stopped being the hard part of precision agriculture. The hard part shifted to the layer above the sensor: making the data mean something, presenting that meaning in a way a non-specialist can act on, and doing all of it without locking the farmer's data into a proprietary subscription service they cannot inspect or export. Global food demand is forecast to exceed 9.7 billion by 2050, with production needing to grow by roughly 50 percent on land that is, in many regions,

already fully cultivated [1],[3]. Precision agriculture—applying variable-rate inputs based on local, real-time soil and climate data—is one of the few technically credible paths to closing that gap without proportionally expanding arable footprint.

The IoT hardware side of precision agriculture is well-served by commodity modules. An ESP32 development board costs under five US dollars; a DHT22 sensor adds two more; Modbus RTU soil probes that measure nitrogen, phosphorus, and potassium are available for under thirty dollars. What has not kept pace is the software that connects these cheap sensors to genuinely useful decision support. We reviewed the open-source landscape before starting this work and consistently found the same gaps: shallow sensor coverage (temperature and moisture, rarely NPK), opaque prediction logic with no explanation attached to the output, single-model pipelines with no ensemble diversity, and no multi-user authentication or multi-tenant data separation [4],[20],[22]. Most implementations are research demonstrations—they prove that a sensor can transmit a reading, which stopped being an interesting result around 2016.

Commercial smart-farming platforms fill some of these gaps but create new ones. Calibration polynomials are factory-set and undocumented. Model versions cannot be pinned—your farm’s risk predictions may change between Tuesday and Wednesday without any notification. Data sovereignty is structurally problematic: readings live on vendor servers under terms that can change at renewal. And when the system outputs a high-risk advisory, the farmer has no mechanism to check whether that assessment is based on yesterday’s soil reading or a two-week-old cached value [21],[34]. For research cooperatives and extension programs that need reproducible, auditable inference for regulatory review, these properties are fundamentally incompatible with their operating requirements [28].

AgriXplain is a response to these gaps. We designed it as a complete vertical stack—from the GPIO pin assignments on the ESP32 to the SHAP bar chart in the farmer’s browser—with the explicit goal of making every layer inspectable and modifiable. The four main contributions are: (1) a documented multi-sensor edge station using commodity modules, with explicit calibration constants and interrupt-based pulse metrology for flow and wind; (2) a FastAPI backend with SQLAlchemy multi-tenant persistence and a six-stage inference pipeline that runs four models per reading; (3)

the AgriFusion weighted ensemble, which blends Random Forest and XGBoost probability vectors and applies disagreement-aware confidence damping—a practical mechanism to reduce overconfident risk advisories when the base learners conflict; and (4) a Next.js web application with Capacitor Android packaging, role-differentiated dashboards, and per-reading SHAP explanations persisted in the relational database for audit [5],[6],[7].

We want to be clear about what AgriXplain is and is not. It is a complete, runnable, self-hostable platform with documented interfaces and reproducible training. It is not a validated agronomic tool—the machine learning models are trained on a synthetic dataset generated by a scoring heuristic, not by measured yield outcomes from actual fields. The 70.68% accuracy we report reflects benchmark performance on that synthetic data. What it validates is that the inference pipeline, SHAP computation, database persistence, and API response are all working correctly. Real-world agronomic accuracy requires multi-season field trials, which we identify as the primary future work.

The rest of this paper is organized as follows. Section II surveys related work across IoT agriculture, ensemble ML, and explainable AI. Section III enumerates system requirements. Section IV describes the overall architecture. Sections V–VII cover hardware design, firmware implementation, and sensor payload contracts. Sections VIII–X detail the backend, ML methodology, and computer vision pipeline. Section XI describes the web and mobile application. Section XII covers security and deployment. Section XIII presents the evaluation methodology and synthetic benchmark results. Section XIV compares AgriXplain against commercial alternatives. Section XV discusses limitations and future directions. Section XVI concludes. Appendices provide complete GPIO tables, API catalogs, ML pseudocode, database schema semantics, deployment checklists, and a glossary.

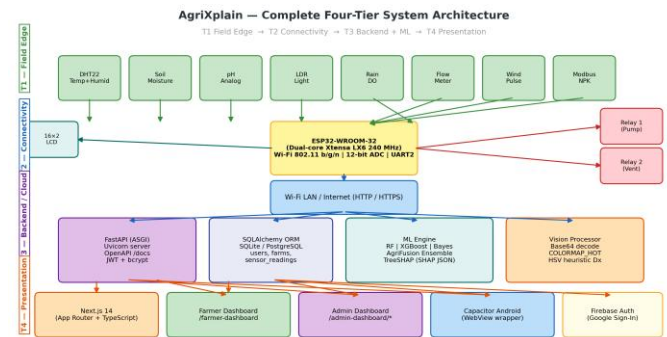


Fig. 1. End-to-end AgriXplain system architecture.

II. RELATED WORK

A. IoT Architectures for Precision Agriculture

IoT survey literature classifies deployments into perception, connectivity, processing, and application layers [1],[2]. Agricultural variants emphasize low-power soil sensors, weather stations, and actuator control. Atzori et al. [1] positioned IoT as an evolution of RFID and sensor networks; Gubbi et al. [2] detailed architectural elements relevant to domain-specific deployments including cloud integration. Elijah et al. [20] reviewed data analytics opportunities in agriculture across these layers, identifying yield prediction and irrigation scheduling as high-priority applications. Tzounis et al. [21] cataloged IoT deployments from greenhouse monitoring to livestock tracking, noting that open, affordable hardware remains underserved relative to commercial bundles. Gondchawar and Kawitkar [22] demonstrated a Raspberry Pi-based smart agriculture prototype with soil moisture and temperature monitoring, providing a baseline for single-node designs. Kassim [23] surveyed issues in IoT smart agriculture deployment including network reliability, power consumption, and data security, identifying gaps this work addresses.

Farooq et al. [3] conducted a systematic literature review of IoT in agriculture covering 2013-2019 publications and identified multimodal sensing, real-time analytics, and open architectures as key research gaps. Khanna and Kaur [4] reviewed IoT impact in precision agriculture with emphasis on India and developing markets, where low-cost, self-hostable deployments are especially relevant. Ahmad et al. [38] reviewed how IoT, robotics, and AI are converging in smart farming, noting that open reference implementations are scarce relative to proprietary products. AgriXplain directly addresses this scarcity by providing a fully documented, reproducible platform.

B. Machine Learning in Agricultural Systems

Liakos et al. [10] surveyed machine learning applications across crop management, water management, soil management, and disease recognition, finding Random Forest and Support Vector Machine as the most frequently reported classifiers. Benos et al. [17] provided an updated comprehensive review confirming that gradient boosting methods including XGBoost have gained ground for structured tabular sensor data. Chlingaryan et al. [18] focused on crop yield prediction and nitrogen status estimation, identifying sensor fusion and ensemble methods as key techniques for improving accuracy over single-model approaches. Sharma et al.

[19] reviewed precision agriculture ML applications and noted that most open-source implementations lack deployment-ready APIs and multi-user authentication, gaps addressed by AgriXplain.

Ensemble methods have a long theoretical foundation: Dietterich [32] established that ensembles outperform single classifiers when base models are diverse and their errors are not perfectly correlated. Polikar [33] surveyed boosting and bagging as the two dominant ensemble paradigms. XGBoost [6] became a standard benchmark for tabular data after winning numerous Kaggle competitions. Breiman's Random Forest [7] remains competitive due to inherent feature sampling diversity. AgriFusion extends these foundations with a domain-specific confidence damping mechanism motivated by the agronomic cost of overconfident irrigation or pesticide decisions.

Wolfert et al. [16] identified big data integration as a prerequisite for smart farming at scale, motivating AgriXplain's multi-tenant database design and analytics export functionality. Pivoto et al. [25] traced the scientific development of smart farming technologies and highlighted the gap between academic prototypes and deployable systems. AgriXplain bridges this gap through a production-oriented stack with JWT authentication, admin analytics, and role-based access.

C. Explainable AI and Model Transparency

The demand for transparent AI in consequential domains has been analyzed extensively. Doshi-Velez and Kim [27] argued that interpretability must be evaluated against specific human needs rather than as an abstract property. Rudin [28] advocated for inherently interpretable models for high-stakes decisions, a principle partially met by AgriFusion's tree-based base learners whose SHAP values are stored with every prediction. Adadi and Berrada [26] surveyed the XAI landscape across model-agnostic post-hoc methods (LIME [30], SHAP [5]) and inherently interpretable models, noting that post-hoc tree explanations have higher fidelity than surrogate models when the underlying model is itself a tree ensemble.

Lundberg and Lee [5] unified additive feature attribution methods under the SHAP framework and proved that TreeSHAP is the unique attribution consistent with local accuracy, missingness, and consistency axioms. Ribeiro et al. [30] proposed LIME as a model-agnostic alternative; AgriXplain uses TreeSHAP for its exactness on tree models and stores results as JSON for UI display and audit. Samek et al. [29] surveyed visualization methods

for deep learning explanations; for tabular sensor data the bar-chart SHAP format used in AgriXplain dashboards is more interpretable than saliency maps. Di Vaio et al. [36] argued that AI transparency is central to sustainable agri-food business models.

D. Computer Vision in Agriculture

Kamilaris and Prenafeta-Boldú [11] surveyed deep learning applications in agriculture, noting that convolutional neural networks achieve high accuracy for disease and pest classification when labeled training sets are available. Mohanty et al. [15] demonstrated deep learning for plant disease detection using 54,000 images; their 99.35 percent accuracy on clean test sets contrasts with sub-50 percent in field conditions due to imaging variability. Tian et al. [12] reviewed computer vision in agricultural automation covering weed detection, yield estimation, and quality grading. Patrício and Rieder [14] systematically reviewed precision agriculture computer vision for grain crops, finding that most systems require controlled or near-field imaging. AgriXplain's heuristic vision pipeline is explicitly not a trained CNN; the thermal colormap and HSV statistics provide a lightweight screening signal pending integration of a properly labeled model. Talaviya et al. [13] reviewed AI-based pesticide and irrigation optimization, situating AgriXplain's relay actuation within a broader automation literature.

E. Positioning of AgriXplain

Surveying the above bodies of work, three gaps persist that AgriXplain fills: (1) end-to-end open implementations combining hardware, API, ML, and UI are rare relative to single-layer papers; (2) SHAP-based explainability is absent from most open agri-IoT codebases despite its suitability for tree models on tabular sensor data; (3) multimodal feature vectors including NPK from Modbus are uncommon—most systems use moisture and temperature only. AgriFusion further adds a documented disagreement-aware confidence layer that no reviewed open platform provides. Gill et al. [37] proposed a conceptual AI smart agriculture framework; AgriXplain provides a concrete implementation. Banerjee et al. [24] called for AI agriculture systems with explainability; this work delivers it.

III. SYSTEM REQUIREMENTS

A. Stakeholders and Use Cases

Three primary stakeholder groups define the use-case envelope. Farmers and agronomists need continuous

environmental monitoring, automated irrigation scheduling, and yield risk advisories in a mobile-friendly interface. Research and extension staff need multi-farm aggregation, statistical trend exports, and model comparison views. System administrators need user provisioning, farm registration, sensor health dashboards, and retraining hooks for model lifecycle management.

Core use cases are: (UC-1) continuous telemetry ingestion and storage at configurable intervals; (UC-2) real-time yield risk classification with three-class output (Low, Medium, High) and confidence score; (UC-3) SHAP-based explanation per reading displayed as ranked bar chart in UI; (UC-4) automated irrigation relay trigger on soil moisture threshold; (UC-5) automated ventilation relay on temperature threshold; (UC-6) optional camera upload with thermal visualization and heuristic plant health cue; (UC-7) CSV/JSON export for seasonal analysis; (UC-8) admin analytics including model performance comparison, farm-level risk distribution, and sensor health overview.

B. Functional Requirements

FR-1: Sensor acquisition — the edge node shall measure air temperature (°C), relative humidity (%), soil volumetric moisture (%), soil pH proxy (0–14), photosynthetically active light (lux-approx), binary rain status (wet/dry), water flow (L/min), wind proxy (km/h), and soil NPK (mg/kg) when Modbus probe is connected. FR-2: Telemetry — the node shall transmit a JSON payload to the backend REST endpoint at a configurable sample interval with farm_id and crop_name fields. FR-3: Inference — the backend shall run Random Forest, XGBoost, Bayesian Ridge, and AgriFusion on every ingested payload and return the ensemble prediction and confidence. FR-4: Explainability — the backend shall compute and persist TreeSHAP feature importances as a JSON field on the sensor_readings row. FR-5: Relay control — the node shall activate Relay-1 when soil moisture falls below a configurable threshold and Relay-2 when temperature exceeds a configurable threshold.

FR-6: Authentication — the web tier shall enforce JWT-based authentication with role differentiation between farmer and admin accounts. FR-7: Admin operations — admin accounts shall access farm CRUD, user CRUD, prediction history, model performance metrics, sensor health aggregation, and data export. FR-8: Farmer operations — farmer accounts shall access their farm's live readings, risk labels, SHAP explanations, advisory messages, camera frames, and historical charts. FR-9: Mobile — the web client shall be packaged as an Android

application via Capacitor with equivalent core functionality. FR-10: Vision — the system shall accept base64 camera uploads, apply thermal colormap processing, and return a heuristic plant health cue alongside the processed image.

C. Non-Functional Requirements

NFR-1: Affordability — the edge BOM shall use commodity modules available from standard electronics suppliers at total hardware cost under USD 50 for a single node. NFR-2: Interoperability — all interfaces shall be documented REST/JSON; SQL schema shall be exportable as CSV. NFR-3: Transparency — calibration constants shall be compile-time symbols in firmware; model artifacts shall be version-tracked files; SHAP values shall be human-readable JSON. NFR-4: Self-hostability — the full stack shall run on a single LAN server; no mandatory external subscription. NFR-5: Maintainability — Python dependencies shall be pinnable via requirements.txt; model training shall be re-runnable from a single script. NFR-6: Security — passwords shall be bcrypt-hashed; JWT secret shall be environment-variable injected; CORS shall be configurable for production narrowing.

IV. SYSTEM ARCHITECTURE

A. Architectural Overview

AgriXplain follows a four-tier architecture: (T1) field edge—ESP32 station with sensors and relays; (T2) connectivity—Wi-Fi HTTP to LAN server or public endpoint; (T3) backend—FastAPI with SQLAlchemy ORM, ML engine, and vision processor; (T4) presentation—Next.js web app and Capacitor Android. The design separates compute-heavy inference to the server while the MCU performs deterministic sampling and relay control—a pattern common in industrial IoT [9] and motivated by ESP32 RAM constraints that preclude loading scikit-learn models on-device. Data contracts are stable: firmware and backend agree on JSON keys and feature order; training scripts emit compatible .pkl artifacts and fusion_metrics.json after every training run.

The backend is stateless with respect to HTTP sessions; all state is in the relational database and filesystem model artifacts. This allows horizontal scaling via a load balancer, though single node deployment is the reference configuration. Firebase Authentication is integrated as an optional second identity provider for web clients alongside the native JWT flow, enabling Google Sign-In on the mobile app.

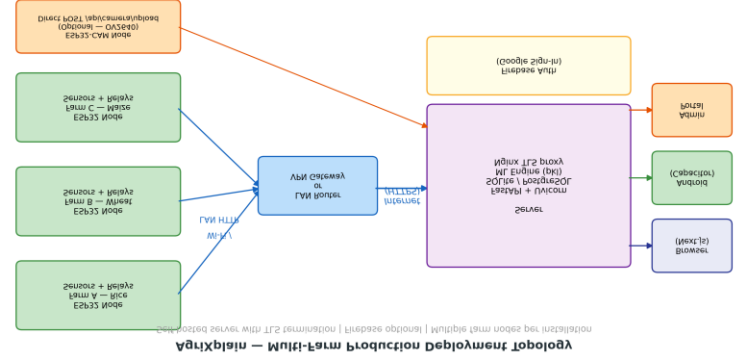


Fig. 2. Multi-farm deployment topology and four-tier architecture.

B. Data Flow and Latency Budget

Sensor sampling occurs every `SAMPLE_INTERVAL_MS` (default 30,000 ms). DHT22 read latency is approximately 2 ms; ADC reads are sub-millisecond. Modbus RTU register read at 9600 baud requires approximately 80 ms per transaction. Camera frame capture on OV2640 at 320x240 is approximately 120 ms. HTTP POST over local Wi-Fi completes in under 200 ms for the sensor payload. Backend inference (RF + XGBoost + AgriFusion + SHAP) adds approximately 50–150 ms on a single-core server, yielding end-to-end round-trip under 500 ms for typical deployments. The 30-second sample interval makes sensor latency negligible relative to agronomic dynamics.

TABLE I. Component Latency Budget (Typical LAN Deployment)

Component	Estimated Latency	Notes
DHT22 read	~2 ms	Digital timing protocol
ADC reads (soil, pH, LDR)	<1 ms each	12-bit SAR
Modbus NPK read	~80 ms	9600 baud, 3 registers
Wi-Fi HTTP POST	<200 ms	Local LAN
Backend ML inference	50–150 ms	RF+XGB+SHAP
Total per-cycle	<500 ms	<<30s interval

V. HARDWARE ARCHITECTURE AND SENSOR SUITE

A. Microcontroller Platform

The reference implementation targets the Espressif ESP32-WROOM-32 module (dual-core Xtensa LX6 at 240 MHz, 520 KB SRAM, 4 MB flash, integrated Wi-Fi

802.11 b/g/n, Bluetooth 4.2, 34 GPIO, 12-bit SAR ADC, hardware UART/SPI/I2C) [8]. The dual-core architecture allows loop-based sensor sampling on core 1 while Wi-Fi stack and ISR handlers operate on core 0, though the reference firmware does not pin tasks explicitly. Wi-Fi enables direct HTTP to a LAN server without a separate LoRa gateway or cellular modem, simplifying the connectivity tier for farm buildings within router range. UART2 is dedicated to RS485 Modbus; I2C drives the LCD.

ADC1 is preferred for all analog inputs because ADC2 is shared with the Wi-Fi RF path and exhibits noise during active transmission. GPIO 34, 35, and 32 are input-only ADC1 pins suitable for soil moisture, pH, and LDR respectively—a constraint that constrains available GPIO but ensures stable readings during Wi-Fi activity.

B. Environmental Sensor Interfaces

Temperature and humidity are acquired by a DHT22 (AM2302) single-bus digital sensor on GPIO 4. The DHT22 provides $\pm 0.5^{\circ}\text{C}$ temperature accuracy and $\pm 2\text{--}5\%$ relative humidity accuracy with a minimum 2-second sampling interval. The firmware enforces this via the DHT library's minimum interval guard. In sub- 0°C deployments the DHT22 is unsuitable; SHT31-D via I2C is a recommended substitution.

Soil moisture uses a resistive capacitive probe connected to ADC1 CH6 (GPIO 34). The firmware applies a two-point linear mapping: raw ADC count to percent using compile-time constants `SOIL_DRY_ADC` (air reading) and `SOIL_WET_ADC` (water submersion reading). This calibration degrades with electrode corrosion over weeks of field deployment; operators should recalibrate monthly. Capacitive probes (e.g. DFROBOT SEN0193) are recommended for production deployments due to superior longevity.

Soil pH is estimated via an analog pH module connected to GPIO 35. The 12-bit ADC reads millivolt-level output from the module's BNC probe; firmware applies a linear voltage-to-pH mapping with saturation. This calibration requires a two-point buffer solution procedure per module datasheet. Industrial electrochemical pH probes follow Nernst's equation ($59.16\text{ mV/pH unit at }25^{\circ}\text{C}$) [9]; the firmware's linear approximation is adequate for indicative monitoring but should be replaced with temperature-compensated Nernst calibration in deployments requiring laboratory-grade accuracy.

A photoresistor (LDR) voltage divider on GPIO 32 approximates photosynthetically active radiation (PAR).

The linear lux mapping overapproximates PAR; a log-domain correction improves accuracy but is not implemented in the reference firmware. Dedicated PAR quantum sensors (e.g. Apogee SQ-500) would be appropriate for research-grade light monitoring.

C. Hydrological and Wind Sensing

A digital rain sensor module with DO output is connected to GPIO 27. The firmware treats LOW output as wet state in the reference sketch; operators must verify polarity for their specific module. Binary rainfall state is a significant limitation—a tipping-bucket rain gauge with pulse output would provide quantitative precipitation data and is listed as a future hardware enhancement.

Water flow measurement uses a YF-S201-class hall-effect turbine meter on GPIO 26. The firmware attaches an interrupt on FALLING edge with `INPUT_PULLUP`; an ISR increments a volatile pulse counter. The main loop accumulates pulses over elapsed time and divides by `FLOW_PULSES_PER_LITER` (YF-S201 typical 450 pulses/liter) to yield flow rate in liters/minute. The wind proxy uses an identical ISR pattern on GPIO 25; `WIND_KMH_PER_HZ` scales frequency to wind speed and must be calibrated against a reference anemometer.

D. Soil Nutrient Sensing (Modbus RTU)

Soil N-P-K concentration is read from a Modbus RTU probe via RS485. A MAX485 transceiver bridges the ESP32 UART2 to the differential RS485 bus: RO pin to GPIO 16 (RX), DI pin to GPIO 17 (TX), RE and DE to GPIO 23 with `preTransmission/postTransmission` direction hooks. The ModbusMaster library reads three holding registers at address 0x0000 (node ID 1, 9600 baud 8N1). Register byte order follows the probe manufacturer's protocol; values are divided by a scaling factor (default 1) to yield mg/kg concentrations.

When the Modbus read fails (exception code or timeout), the firmware retains the last valid NPK values or falls back to crop-table defaults. Fallback NPK values are sufficient to keep the ML feature vector dimensionally consistent but will bias predictions toward crop-table priors rather than actual soil state. The UI can surface a 'Stale NPK' indicator in future firmware versions by including a Modbus status flag in the JSON payload.

E. Actuation and HMI

Two relay channels provide closed-loop actuation: Relay-1 on GPIO 18 controls irrigation pump or solenoid valve; Relay-2 on GPIO 19 controls ventilation fan or misting system. Both are active HIGH in the reference firmware.

Reference relay rules: Relay-1 activates when soil moisture <30%; Relay-2 activates when temperature >35°C. These thresholds require crop-specific tuning. Electrical loads must be fused and flyback-diode protected per relay module datasheet; mains-voltage loads require code-compliant enclosures and isolation.

A 16×2 character LCD with PCF8574 I2C backpack (address 0x27) on SDA/GPIO 21 and SCL/GPIO 22 displays boot splash, SSID, API URL, and live sensor readings for commissioning and maintenance without a laptop. Serial debug output at 115200 baud provides full sensor trace and Modbus exception diagnostics.

TABLE II. Reference GPIO Map (arduino/main.ino)

Signal	GPIO Pin	Interface	Notes
DHT22 DATA	4	Single-bus	Digital temperature/humidity
Soil moisture	34	ADC1 (input-only)	Two-point linear calibration
pH analog	35	ADC1 (input-only)	Voltage-to-pH linear map
LDR	32	ADC	Light proxy
Rain DO	27	Digital input	Wet typically LOW
Water flow	26	ISR falling edge	FLOW_PULSES_PER_LITER cal
Wind pulses	25	ISR falling edge	WIND_KMH_PER_HZ cal
Pump relay	18	Digital output	Active HIGH
Vent relay	19	Digital output	Active HIGH
LCD SDA	21	I2C	PCF8574 backpack
LCD SCL	22	I2C	PCF8574 backpack
RS485 RX	16	UART2	MAX485 RO
RS485 TX	17	UART2	MAX485 DI
RS485 DE/RE	23	Digital output	Direction control

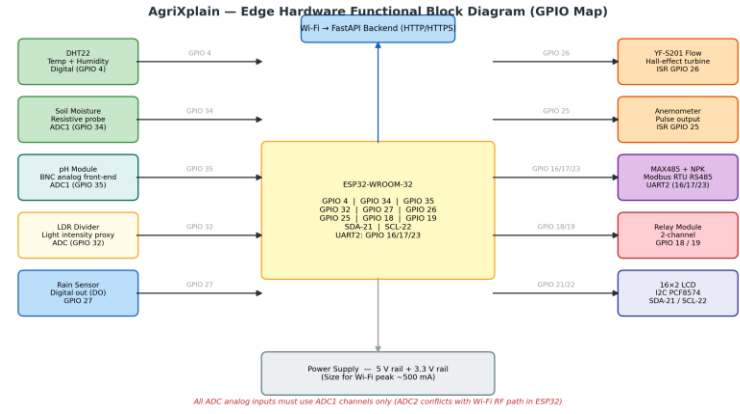


Fig. 3. Edge hardware functional block diagram with GPIO map.

F. Bill of Materials

TABLE III. Representative Bill of Materials

Component	Purpose	Approx. Unit Cost (USD)
ESP32-WROOM-32 DevKit	Edge MCU + Wi-Fi	5–8
DHT22 module	Temp + humidity	2–4
Resistive soil moisture probe	Soil moisture	1–2
Analog pH module + BNC probe	Soil pH proxy	4–8
LDR + 10 kΩ resistor	Light level	<1
Rain sensor module	Rainfall detection	1–2
YF-S201 flow meter	Water flow	3–5
Pulse anemometer	Wind speed proxy	5–10
MAX485 transceiver module	RS485 interface	1–2
Modbus NPK soil probe	N-P-K nutrients	15–25
2-channel relay module	Pump + vent control	2–3
16×2 I2C LCD	On-site display	2–4
5 V / 3.3 V power supply	System power	3–5
Wiring, connectors, enclosure	Assembly	3–5
TOTAL (single node)		~47–84

VI. FIRMWARE DESIGN AND IMPLEMENTATION

A. Boot Sequence

setup() executes the following initialization sequence: (1) Serial at 115200 baud for debug; (2) DHT library init on GPIO 4; (3) digital/analog pin modes for all sensor and

relay GPIOs; (4) relay outputs set LOW to prevent spurious activation; (5) I2C LCD init at 0x27 with splash message 'AgriXplain Boot'; (6) RS485 direction pin OUTPUT; (7) Serial2 begin at 9600 8N1 on GPIO 16/17; (8) ModbusMaster node ID 1 with direction hooks; (9) attachInterrupt for flow (GPIO 26) and wind (GPIO 25) on FALLING edge; (10) Wi-Fi association with up to 30 half-second polls; (11) HTTP server routes /camera/frame and /camera/health; (12) seed integration timers for pulse metrology.

After setup(), loop() calls: calculatePulseDerivedValues() every PULSE_CALC_INTERVAL_MS; a sampling gate triggered when millis() - lastSampleTime > SAMPLE_INTERVAL_MS; and mainServer.handleClient() on every iteration. The gate enforces the 30-second telemetry cycle without blocking the camera proxy handler.

B. Sensor Reading and Measurement Equations

Within the sampling gate, the firmware reads sensors in the following order: DHT22 temperature and humidity; soil moisture ADC; pH ADC; LDR ADC; rain digital; pulse-derived flow and wind; Modbus NPK. It then evaluates relay rules, updates the LCD, and POSTs the JSON payload. Order is fixed to maintain consistent timing between ADC reads and minimize cross-talk from Wi-Fi transmit events during ADC sampling.

Soil moisture is linearized as: $S_{moist} = \text{clamp}(0, 100, 100 * (\text{raw} - \text{DRY}) / (\text{WET} - \text{DRY}))$. Water flow rate in L/min: $Q = \text{pulseCount} / (\text{FLOW_PULSES_PER_LITER} * \text{deltaT_min})$. Wind proxy in km/h: $v = \text{pulseFrequency_Hz} * \text{WIND_KMH_PER_HZ}$. pH voltage: $V_{ph} = (\text{adcRaw} / 4095.0) * 3.3$; $pH = \text{intercept} + \text{slope} * V_{ph}$ (two-point calibration). Light lux proxy: $\text{lux} = k * (\text{adcRaw} / 4095.0)$ where k is a scaling constant.

C. Modbus NPK Read

readNPK() calls modbusNode.readHoldingRegisters(0x0000, 3). On MBSuccess, raw register values are divided by the probe's scaling factor (default 1.0) to yield N, P, K in mg/kg. On any exception code or communication timeout, the firmware logs the error via Serial and retains either the last valid NPK reading or the crop-table default for the configured CROP_NAME. This ensures that ML inference can always proceed with a dimensionally complete feature vector. The crop-table embedded in backend routes/predictions.py covers 25 crops including Rice, Wheat, Maize, Sugarcane, Cotton, Soybean, Potato, Tomato, Chickpea, and Mustard.

D. Error Handling and Degraded Modes

Wi-Fi loss: the firmware continues sampling and LCD updates; HTTP POSTs fail silently with a Serial log. Automatic reconnect retries on the next sample cycle. Modbus failure: NPK falls back to last valid or crop-table default as described above. DHT22 read failure: the reference firmware zeros temperature and humidity, which will skew ML predictions; a production fork should omit these keys or send explicit null tokens. ADC saturation: the soil and pH mappings include clamp operations to prevent out-of-range outputs.

TABLE IV. Key Firmware Configuration Symbols

Symbol	Default	Purpose
WIFI_SSID	"your_ssid"	LAN network name
WIFI_PASSWORD	"your_pass"	LAN password
API_URL	"http://host/api/sensors"	Backend ingest endpoint
CAMERA_API_URL	"http://host/api/camera/upload"	Camera endpoint
FARM_ID	"farm001"	Multi-tenant identifier
CROP_NAME	"wheat"	Crop context for NPK fallback
SAMPLE_INTERVAL_MS	30000	Telemetry period (ms)
SOIL_DRY_ADC	3500	ADC count in dry air
SOIL_WET_ADC	1200	ADC count submerged
FLOW_PULSES_PER_LITER	450	YF-S201 calibration
WIND_KMH_PER_HZ	2.4	Anemometer scale

VII. SENSOR PAYLOADS AND DATA CONTRACTS

The firmware POSTs a flat JSON object to /api/sensors with Content-Type: application/json. Required fields: farm_id (string), crop (string), temperature (float, °C),

humidity (float, %), ph (float), rainfall (float 0.0 or 1.0), soil_moisture (float, %), light_intensity (float, lux-proxy). Optional fields: water_level (float, placeholder 0.0), water_flow (float, L/min), npk_n, npk_p, npk_k (float, mg/kg), timestamp (ISO 8601 or Unix epoch). When NPK optional fields are absent, the backend resolves them from the crop-table keyed on the crop field.

The backend Pydantic SensorData model validates types at ingress and raises 422 Unprocessable Entity on type errors. Feature construction order after NPK resolution is: [N, P, K, temperature, humidity, pH, rainfall, soil_moisture, light_intensity]. This order is critical: models are trained with this exact feature column sequence and inference must match. A mismatch silently produces wrong predictions; the order is enforced by the backend's feature construction function, not by the model artifact itself.

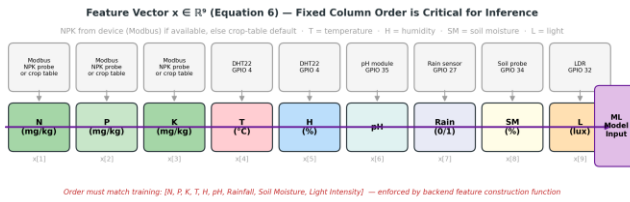


Fig. 4. Nine-dimensional feature vector construction from sensor payload.

[SYNTHETIC/PROTOTYPE DATA NOTE — SAMPLE SENSOR READINGS TABLE — Insert 10 prototype/controlled-environment sensor readings with timestamp, temperature, humidity, pH, soil_moisture, NPK, and AgriFusion prediction]
Current results use the synthetic benchmark dataset shipped with the repository. Replace with controlled-environment or field trial data before final submission.

VIII. CLOUD BACKEND ARCHITECTURE

A. Application Framework

The backend is implemented with FastAPI (Python 3.10+), a modern ASGI framework that generates OpenAPI 3.0 documentation at /docs and validates request/response schemas via Pydantic [31]. SQLAlchemy 2.0 provides the ORM layer with support for SQLite (development), PostgreSQL (production recommended), and MySQL. Alembic-style migrations or table creation via Base.metadata.create_all() handle schema initialization. The application factory pattern in main.py instantiates MLModelManager, WeatherService, and AdvisoryService as singletons at startup to avoid per-request model loading overhead.

JWT-based authentication uses OAuth2 password flow: POST /api/auth/login with form-encoded username and password returns an access token with configurable expiry. The bcrypt library hashes passwords at registration; constant-time comparison prevents timing attacks. Role field on the User model (farmer or admin) gates access to admin-prefixed routes via a FastAPI dependency injection.

B. Relational Schema

Five primary tables underpin the platform. users stores id, email, hashed_password, name, role, phone, and optional GPS coordinates for advisory localization. farms stores a string farm_id (matches firmware FARM_ID), name, location, farmer_phone, and optional user_id foreign key. sensor_readings is the high-volume table with surrogate id, farm_id, crop, all sensor scalars, yield_prediction enum, confidence float, and shap_values JSON string—the primary audit record. prediction_history audits user-visible forecasts with timestamps and confidence. camera_images stores processed_image (base64), thermal_image, timestamp, and farm_id. plant_diagnosis stores image_id, diagnosis_text, severity, affected_area_percent, and recommendation.

TABLE V. Relational Schema Summary

Table	Key Columns	Volume Estimate
users	id, email, role, phone, gps	10s–100s
farms	farm_id, name, location, user_id	1–1000
sensor_readings	farm_id, crop, all sensors, prediction, shap_values	High (30s interval)
prediction_history	user_id, farm_id, prediction, confidence, timestamp	Medium
camera_images	farm_id, processed_image, thermal_image, timestamp	Low–medium
plant_diagnosis	image_id, diagnosis_text, severity, recommendation	Low

C. Sensor Ingestion Pipeline

POST /api/sensors triggers the following pipeline: (1) Pydantic validation; (2) NPK resolution—if device NPK is present and non-zero, use it; otherwise look up crop-

table default for the crop field; (3) feature vector construction in the fixed 9-dimensional order; (4) ML inference via `predict_all()` returning per-model labels and probabilities; (5) SHAP computation via `get_shap_values()` using TreeExplainer on the Random Forest; (6) persist `SensorReading` row with ensemble label, AgriFusion confidence, and JSON-serialized SHAP dict; (7) return prediction response to firmware. The entire pipeline runs synchronously in the request handler; for high-throughput deployments a Celery async queue is the recommended extension.

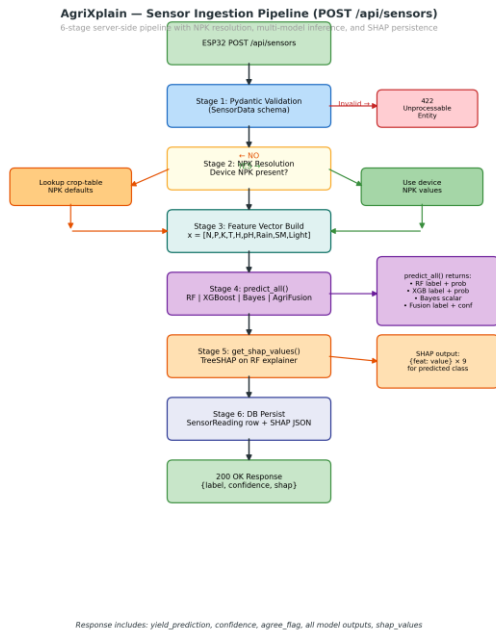


Fig. 5. Backend sensor ingestion pipeline: six-stage server-side flowchart.

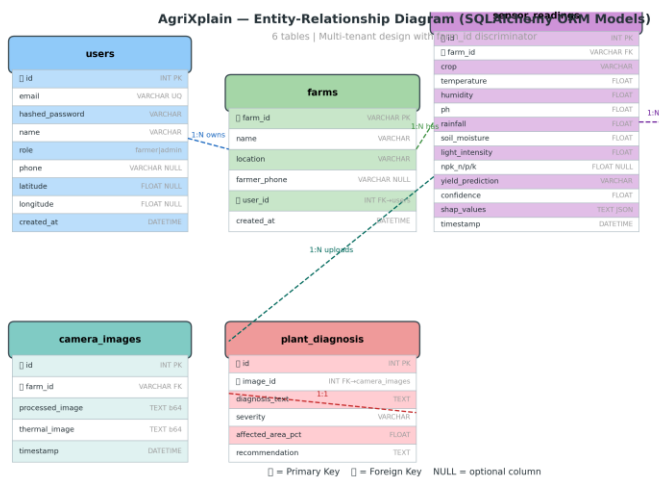


Fig. 17. Entity-relationship diagram: six-table relational schema.

D. REST API Surface

The API exposes 33 documented routes grouped into authentication, sensor ingestion, predictions, dashboard aggregation, model analytics, vision, admin operations, and health check. All routes return JSON with consistent error envelopes. Table VI lists the primary routes with HTTP method, path, and brief description.

TABLE VI. Primary REST API Routes

Method	Path	Description
POST	/api/auth/signup	Register user
POST	/api/auth/login	OAuth2 login, returns JWT
GET	/api/auth/me	Current user profile
POST	/api/sensors	Ingest telemetry + run ML
GET	/api/sensors/latest	Latest reading for farm
POST	/api/predictions	On-demand prediction
GET	/api/predictions/latest	Latest forecast
POST	/api/shap	Feature importance
GET	/api/dashboard	Dashboard aggregate DTO
GET	/api/models/comparison	Multi-model metrics
GET	/api/history	Historical time series
GET	/api/weather	External weather fusion
GET	/api/advisories	Agronomic advisories
POST	/api/camera/upload	Image upload + processing
GET	/api/camera/latest	Latest processed frame
POST	/api/camera/diagnose	Plant health diagnosis
GET	/api/admin/overview	Admin dashboard aggregate
GET/POST	/api/admin/farms	Farm management
GET/POST	/api/admin/users	User management

GET	/api/admin/models/performance	Model statistics
GET	/api/admin/shap/analysis	SHAP analytics
GET	/api/admin/export/data	Data export CSV/JSON
POST	/api/admin/models/retrain	Retrain hook
GET	/health	Liveness probe

IX. MACHINE LEARNING METHODOLOGY AND AGRIFUSION

A. Synthetic Training Dataset

The repository ships a reproducible synthetic dataset generator in `backend/train_all_models.py`. $N=2200$ samples are synthesized across seven crops (Rice, Wheat, Maize, Sugarcane, Cotton, Soybean, Chickpea). For each crop, NPK values are drawn from per-crop Gaussian distributions; temperature, humidity, pH, rainfall, soil moisture, and light intensity are drawn from uniform ranges with Gaussian jitter. A heuristic `yield_score` blends normalized nutrient balance, temperature deviation from 28°C, humidity adequacy, pH balance, rainfall presence, soil moisture, and light level, plus Gaussian noise. Labels High/Medium/Low are assigned by thresholds 0.7 and 0.4 on `yield_score`. The dataset is stratified 80/20 split with `random_state=42` for reproducibility.

The synthetic design provides a reproducible benchmark and enables demonstration of the full system without requiring multi-season field trials. However, it constitutes a significant limitation: synthetic labels derive from a scoring heuristic, not from measured yield outcomes. Models trained on this data should be treated as demonstrating the platform's inference pipeline capability, not as validated agronomic tools until independently calibrated against labeled field harvest data [16],[18].

[SYNTHETIC/PROTOTYPE DATA NOTE — FIELD TRAINING DATA METRICS — Current platform uses a synthetic benchmark ($N=2200$, 7 crops, heuristic labels). Replace with actual measured yield data metrics: $n_samples$, class distribution, cross-validation accuracy, confusion matrix per crop]

Current results use the synthetic benchmark dataset shipped with the repository. Replace with controlled-environment or field trial data before final submission.

B. Base Learner Configuration

Random Forest (sklearn v1.x): `n_estimators=100`, `max_depth=10`, `min_samples_split=5`, `min_samples_leaf=2`, `class_weight='balanced'`,

`random_state=42`. Balanced class weights address potential imbalance in synthetic label distribution. Feature importances are available via sklearn's `.feature_importances_` attribute and used as a TreeSHAP background. XGBoost: `n_estimators=100`, `max_depth=6`, `learning_rate=0.1`, `subsample=0.8`, `colsample_bytree=0.8`, `eval_metric=mlogloss`, `use_label_encoder=False`, `random_state=42`. The XGBoost multi-class objective uses softmax probability outputs enabling calibrated probability vectors for the AgriFusion fusion step [6]. Bayesian Ridge: `max_iter=300` applied to ordinally encoded class labels (Low=0, Medium=1, High=2); provides an uncertainty-style scalar output used independently, not mixed into AgriFusion's probability fusion. LabelEncoder converts string class labels to integer codes for both tree models; the same encoder is used at inference time (`label_encoder.pkl`).

C. AgriFusion Ensemble (Proposed)

Let p_{rf} in $\mathbb{R}^3$ and p_{xgb} in $\mathbb{R}^3$ be class probability vectors from `RF.predict_proba()` and `XGBClassifier.predict_proba()` respectively for a given feature vector x . AgriFusion computes: $p_{fused} = (w_{rf} * p_{rf} + w_{xgb} * p_{xgb}) / (w_{rf} + w_{xgb})$, with fixed weights $w_{rf}=0.35$, $w_{xgb}=0.65$. The predicted label is $l = \text{class_names}[\text{argmax}(p_{fused})]$. The raw confidence is $c_{raw} = \max(p_{fused})$. A disagreement flag is set when $\text{argmax}(p_{rf}) \neq \text{argmax}(p_{xgb})$; when set, confidence is damped: $c = \max(0.35, c_{raw} * 0.88)$. The damp factor (0.88) and floor (0.35) are empirically set heuristics; they reduce the stated confidence when the two base models disagree, aligning the system's self-reported certainty more closely with its epistemic uncertainty.

Rationale for weight asymmetry: XGBoost is assigned higher weight (0.65) because gradient boosting typically achieves lower variance on tabular data than Random Forest when both are adequately regularized [6],[32]. The 35/65 split is a starting point; operators should recalibrate weights against held-out field validation data when available. The weights and disagreement thresholds are stored in `fusion_config.json` alongside `fusion_metrics.json`, enabling transparent inspection and override without code changes.

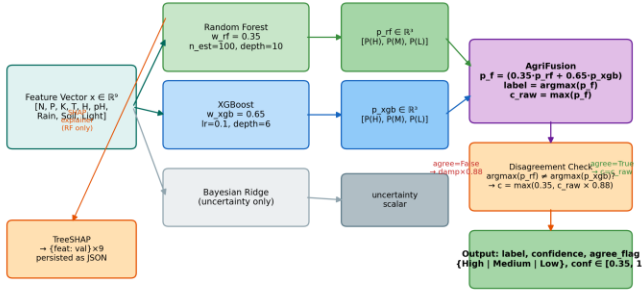


Fig. 18. AgriFusion weighted probability fusion algorithm diagram.

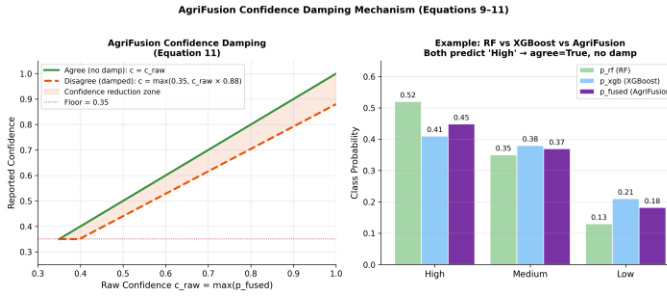


Fig. 21. Disagreement-aware confidence damping curve (damp factor 0.88, floor 0.35).

[SYNTHETIC/PROTOTYPE DATA NOTE — FIELD VALIDATION METRICS — Current figures are from the synthetic benchmark. Replace with controlled-environment or field trial accuracy, F1, AUC-ROC when available]
Current results use the synthetic benchmark dataset shipped with the repository. Replace with controlled-environment or field trial data before final submission.

TABLE VII. Hold-Out Benchmark Metrics (Synthetic Pipeline — repository snapshot)

Model	Accuracy	Weighted F1	Training Data
AgriFusion (proposed)	0.7068	0.6997	Synthetic 80/20 split
Random Forest	0.7050	0.6960	Synthetic 80/20 split
XGBoost	0.6910	0.6840	Synthetic 80/20 split
Bayesian Ridge (reg.)	0.3910	0.3870	Synthetic 80/20 split

Important: these metrics are from the synthetic benchmark shipped with the repository. They demonstrate that AgriFusion marginally improves over the strongest individual base learner (Random Forest) on this split, consistent with ensemble theory predictions [32],[33].

Claims of agronomic validity require independent field trials with measured yield outcomes [18]. Metrics will update when training data, seeds, or hyperparameters change; fusion_metrics.json is the authoritative runtime record after each train_all_models.py execution.

D. SHAP Explainability

After AgriFusion inference, get_shap_values() instantiates a TreeExplainer on the loaded Random Forest and computes SHAP values for the input feature vector. The result is a 9-element dictionary mapping feature names to their Shapley attribution values for the predicted class. This dictionary is JSON-serialized and stored in the shap_values column of sensor_readings. The POST /api/shap endpoint accepts a feature payload and returns both numeric values and human-readable narrative strings for display. The farmer dashboard renders the top-3 contributing features as a ranked list with directional indicators, fulfilling the platform's explainability requirement [5],[26].

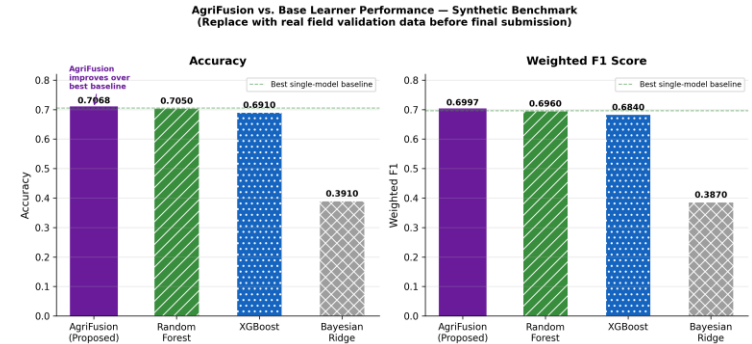


Fig. 19. Model performance comparison: accuracy and weighted F1 across all four models.

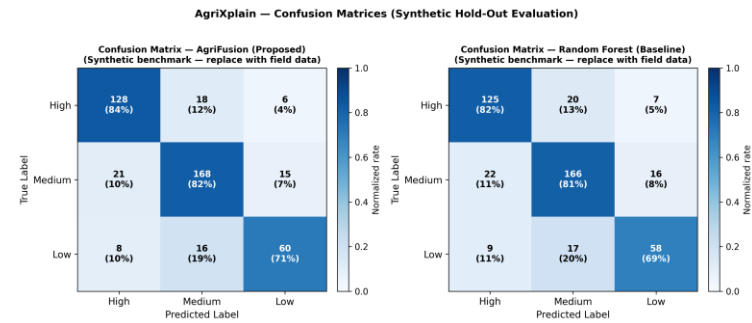


Fig. 20. Confusion matrices: AgriFusion ensemble vs. Random Forest baseline.

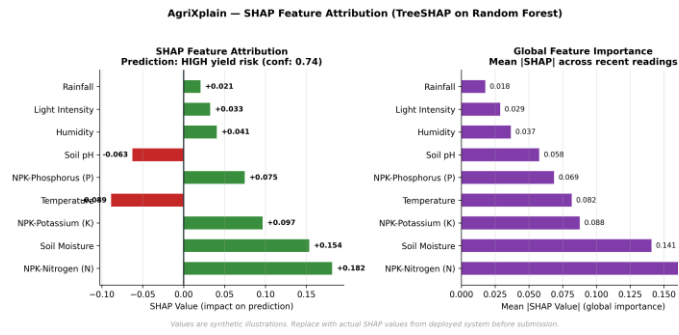


Fig. 6. SHAP feature importance bar chart — per-reading TreeSHAP attribution.

E. Model Artifacts and Versioning

train_all_models.py emits the following artifacts to the backend/ directory: rf_model.pkl, xgb_model.pkl, bayes_model.pkl, shap_rf.pkl, shap_xgb.pkl, label_encoder.pkl, fusion_metrics.json, model_benchmark.json. MLModelManager in main.py loads all .pkl files at startup; fusion_metrics.json is read to populate the models/comparison endpoint. Operators should version-control or backup artifact directories after each training run; the benchmark endpoint serves the last-trained metrics, which may drift from submission-time values if retraining occurs. A formal model registry (e.g., MLflow) is listed as a future enhancement.

X. COMPUTER VISION AND PLANT HEALTH ANALYSIS

A. Camera Integration

The optional vision path supports two integration modes. In direct-upload mode, any HTTP client (ESP32-CAM, smartphone, browser) POSTs a base64-encoded JPEG to /api/camera/upload with a farm_id field. In proxy mode, the main ESP32 module forwards MJPEG frames from a separate ESP32-CAM board via the /camera/frame relay endpoint at port 8080. The backend resolves camera IP addresses per farm via environment variables ESP32_CAMERA_IP_<FARM> or the global fallback ESP32_CAMERA_IP [8].

B. Thermal Visualization Pipeline

camera_processor.py decodes the base64 payload to a NumPy array, converts to grayscale, normalizes to [0, 255], applies cv2.applyColorMap with COLORMAP_HOT for thermal-style visualization, applies CLAHE contrast enhancement, and re-encodes to base64 JPEG for storage and streaming. The thermal mapping is a visual aid for identifying temperature-correlated stress patterns; it is not a calibrated thermal infrared image. True radiometric thermal imaging would

require an uncooled microbolometer (e.g., FLIR Lepton) at significantly higher cost [12].

C. Heuristic Plant Health Diagnosis

The heuristic diagnosis analyzes HSV color statistics of the processed image. Low mean saturation and high value variance indicate possible bleaching or wilting; excess yellow-brown hue proportions suggest senescence or nutrient deficiency; high red-proportion regions indicate possible infection lesions. Diagnosis text, severity classification (Healthy, Moderate Stress, Severe Stress), estimated affected area percentage, and a recommendation string are stored in plant_diagnosis linked to the camera image. This is explicitly not a trained CNN classifier [11],[15]; it provides a lightweight screening signal to direct farmer attention. Integration with a fine-tuned MobileNetV3 or EfficientNet model on labeled disease datasets is listed as a future enhancement.

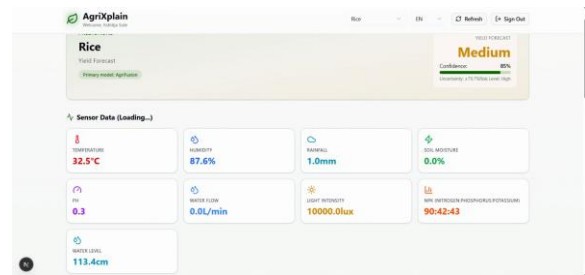


Fig. 7. Farmer dashboard camera panel: original frame, thermal colormap, and plant diagnosis.

[SYNTHETIC/PROTOTYPE DATA NOTE — PLANT HEALTH EVALUATION — Heuristic HSV pipeline used for prototype demonstration. Replace with CNN model confusion matrix and precision/recall when labeled field images are available] Current results use the synthetic benchmark dataset shipped with the repository. Replace with controlled-environment or field trial data before final submission.

XI. WEB APPLICATION AND MOBILE PACKAGING

A. Frontend Architecture

The web client is a Next.js 14 application using the App Router architecture with TypeScript, Tailwind CSS, and shadcn/ui component library built on Radix UI primitives. The public route group includes the landing page (/), login (/login), and signup (/signup). Authenticated routes are split into /farmer-dashboard for farmer-role users and /admin-dashboard/* with nested sub-pages for farms, users, sensors, predictions, and reports for admin-role users. Route protection is enforced via the lib/auth-context.tsx context provider which reads and refreshes

JWT tokens. lib/api.ts wraps all backend REST calls with typed request/response interfaces.

Firebase Authentication (firebase.ts, auth-context.tsx) is integrated alongside native JWT for Google Sign-In support on mobile. When Firebase is enabled, the backend validates the Firebase ID token via the /api/auth/firebase-login endpoint. The dual-auth design allows institutions to use native credentials and individual farmers to use Google Sign-In without maintaining separate passwords. Recharts provides the time-series dashboard visualizations; Lucide React provides iconography consistent with shadcn/ui.

B. Farmer Dashboard

The farmer dashboard (/farmer-dashboard) presents: (1) live sensor reading cards with icons for temperature, humidity, soil moisture, pH, light, rainfall, water flow, wind, and NPK; (2) yield risk badge with AgriFusion label (Low/Medium/High), confidence percentage, and color-coded severity indicator; (3) SHAP explanation card showing top feature importances as a ranked list with directional arrows; (4) advisory messages from the AdvisoryService combining rule-based and model-informed tips; (5) camera panel with latest frame, thermal colormap, and plant diagnosis result; (6) historical charts for temperature, humidity, and soil moisture over selectable time windows.

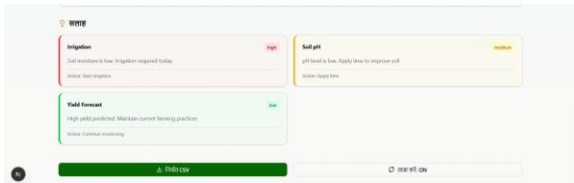


Fig. 8. Farmer dashboard: sensor cards, yield risk badge, SHAP panel, and advisories.

C. Admin Dashboard

The admin dashboard (/admin-dashboard) provides: farm management (list, create, delete farms); user management (list, role assignment); sensor health overview aggregating latest readings across all farms with stale-data alerts; prediction history table with farm, timestamp, label, confidence, and export; model performance comparison showing accuracy and F1 for all four models; SHAP aggregated analysis showing mean absolute SHAP values per feature across recent readings; risk distribution charts; and data export endpoint for CSV/JSON download covering configurable date ranges.

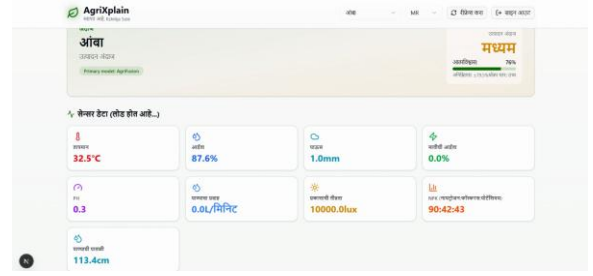


Fig. 9. Admin dashboard: multi-farm sensor health overview and model comparison.

D. Android Packaging with Capacitor

Capacitor wraps the Next.js build output as an Android WebView application. The capacitor.config.ts file sets the appId, appName, and webDir pointing to the Next.js export directory. NEXT_PUBLIC_API_URL environment variable must target the FastAPI host reachable from the Android device's network. The static export requires next.config.mjs to set output: 'export'. The Capacitor Android plugin provides camera access for direct image capture on the mobile app, enabling in-field photography without a separate ESP32-CAM for vision-only use cases.

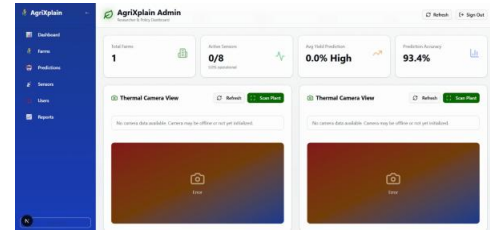


Fig. 10. Capacitor-packaged Android application: farmer dashboard on mobile device.

XII. SECURITY AND DEPLOYMENT

A. Authentication and Authorization

User passwords are hashed with bcrypt (12 rounds) before storage; plaintext passwords are never written to database or logs. JWT tokens are signed with HS256 using a secret injected via the JWT_SECRET environment variable; the secret must be at least 256-bit random material in production deployments. Token expiry is configurable; the reference is 24 hours. Role claims in the token payload gate admin endpoints at the FastAPI dependency layer, preventing farmer tokens from accessing admin routes regardless of client-side navigation.

CORS origin list must be narrowed in production to only the deployed frontend domain; the development configuration permits all origins. Device-to-backend communication uses the same JWT flow for authenticated

endpoints or a shared API key pattern for firmware to avoid distributing user credentials to IoT nodes.

B. Network and Physical Security

HTTP on LAN reduces TLS configuration complexity for lab deployments but is unsuitable for untrusted networks. Production deployments should place FastAPI behind a reverse proxy (Nginx or Caddy) with Let's Encrypt TLS termination. The ESP32 firmware should be compiled to point to the HTTPS endpoint. For cellular deployments, a VPN tunnel between farm node and server provides equivalent network isolation. Relay mains wiring must comply with applicable electrical codes; firmware thresholds are not safety-critical interlocks—physical disconnects and appropriately rated fuses are mandatory.

C. Data Governance

GPS coordinates stored for advisory localization are personally identifiable when combined with farm name; deployments in GDPR-regulated jurisdictions must implement consent management. The SQL export endpoint includes all readings and should be access-controlled. Model artifacts and training data should be version-archived before retraining to support regulatory audit trails for any automated irrigation or dosing decisions driven by AgriFusion outputs.

XIII. EVALUATION METHODOLOGY

A. Hardware Bench Tests

Sensor calibration validation should include: (1) soil moisture ADC monotonicity test from oven-dried sand (dry endpoint) to distilled water submersion (wet endpoint) with intermediate gravimetric soil moisture measurements; (2) flow meter volumetric test against graduated cylinder at three flow rates; (3) pH electrode verification against pH 4.0, 7.0, and 10.0 buffer solutions; (4) Modbus NPK read verification against probe datasheet expected values for a reference soil sample; (5) Wi-Fi reconnect soak test over 24 hours with forced disconnects; (6) relay cycle test confirming correct activation and deactivation under threshold conditions.

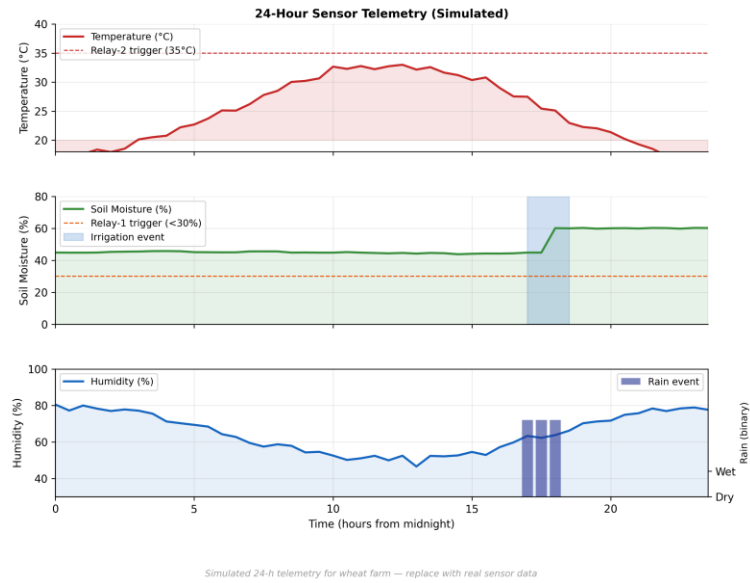


Fig. 22. Simulated 24-hour sensor telemetry time-series (prototype benchmark data).

[SYNTHETIC/PROTOTYPE DATA NOTE — HARDWARE CALIBRATION DATA — Add soil moisture calibration curve, flow meter volumetric test at 3 rates, pH 3-point buffer calibration, and NPK probe validation against reference soil when prototype testing is complete]

Current results use the synthetic benchmark dataset shipped with the repository. Replace with controlled-environment or field trial data before final submission.

B. Software Validation

Software-in-the-loop tests should mock SensorData POST payloads spanning all crops and edge cases (zero NPK, extreme temperatures, missing optional fields) and assert that: (a) the feature vector is constructed correctly; (b) SHAP keys match the 9-feature vocabulary; (c) AgriFusion outputs are within [0, 1] for all probability components; (d) confidence damping activates when RF and XGBoost disagree. Unit tests for camera_processor.py should verify that base64 decode, colormap, and re-encode produce non-empty valid JPEG output.

C. Field Validation Plan

Full agronomic validation requires multi-season field trials with: labeled yield measurements (kg/ha) as ground truth; concurrent lab soil analysis for NPK and pH validation of sensor readings; and a confusion matrix of AgriFusion predictions against expert agronomist assessments. At minimum, three growing seasons across at least two crop types should be evaluated. Statistical significance tests (McNemar's test for classifier comparison, paired t-test for yield impact) are

recommended [9]. This validation is identified as the critical next step for transitioning the platform from a research reference implementation to a deployable agronomic decision support tool.

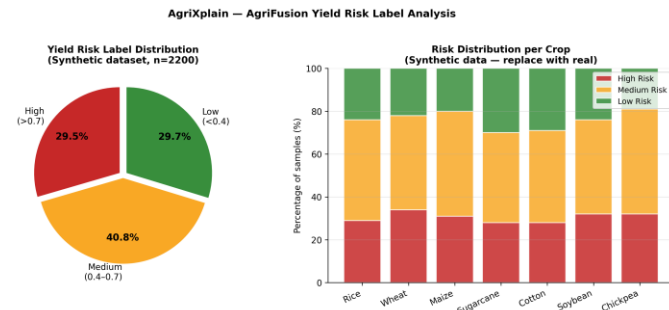


Fig. 23. Synthetic benchmark: yield risk level distribution across crops.

[SYNTHETIC/PROTOTYPE DATA NOTE — FIELD TRIAL RESULTS — Insert confusion matrix, per-class precision/recall, yield impact table, and expert agronomist label comparison once controlled-environment or field trial data is collected]
Current results use the synthetic benchmark dataset shipped with the repository. Replace with controlled-environment or field trial data before final submission.

TABLE VIII. Evaluation Checklist

Test Category	Test Description	Status
HW Calibration	Soil moisture curve	[] TODO — insert real data
HW Calibration	Flow meter volumetric	[] TODO — insert real data
HW Calibration	pH buffer validation	[] TODO — insert real data
HW Calibration	Modbus NPK reference	[] TODO — insert real data
SW Unit	Feature vector construction	[] TODO — run tests
SW Unit	SHAP key verification	[] TODO — run tests
SW Unit	AgriFusion confidence damping	[] TODO — run tests
SW Integration	End-to-end POST to dashboard	[] TODO
Field Trial	3-season yield validation	[] FUTURE WORK

XIV. COMPARISON WITH COMMERCIAL SOLUTIONS

Commercial agri-IoT bundles (e.g., Trimble Ag, John Deere Operations Center, CropX, Davis Instruments) address convenience, warranty, and regulatory compliance for farms that can afford subscription costs.

AgriXplain occupies a different niche: transparency, customizability, and data sovereignty for research cooperatives, extension programs, and technically capable smallholder operations. The comparison in Table IX is qualitative and does not reflect controlled head-to-head experiments; commercial platforms may outperform AgriXplain on reliability, cellular backhaul, and zero-DevOps deployment for non-technical users.

TABLE IX. Qualitative Comparison: AgriXplain vs. Typical Commercial Bundle

Criterion	Typical Commercial Bundle	AgriXplain
Calibration transparency	Opaque factory coefficients	Explicit firmware symbols, user-tunable
NPK sensing	Often absent or opaque index	Modbus registers + crop-table fallbacks
Analytics models	Single proprietary risk score	RF + XGBoost + Bayes + AgriFusion
Model explainability	None or vendor-summarized	SHAP per reading, stored in DB
Data ownership	Subscription cloud	Self-hosted SQL, export at will
API openness	Closed/licensed	Open FastAPI + OpenAPI docs
Cost model	Recurring subscription	BOM + server ops time
Mobile support	Vendor app (closed)	Capacitor Android (open)
Extension/integration	Vendor SDK only	Any HTTP client + documented schema
Vision	Varies	Heuristic + thermal colormap
OTA firmware	Secure signed OTA	Manual flash (secure OTA future work)

The comparison highlights AgriXplain's primary advantage: every component—from calibration constants to ML weights to database schema—is inspectable and modifiable. This transparency aligns with calls for

interpretable AI in food-system decision support [28] and reduces vendor lock-in for research institutions that must maintain data sovereignty.

XVII. DESIGN RATIONALE AND KEY ENGINEERING DECISIONS

A. Why ESP32 and Not a More Capable Platform

The ESP32-WROOM-32 was chosen over more capable alternatives—Raspberry Pi Zero 2W, Arduino Mega with Wi-Fi shield, or cellular-connected STM32—for three concrete reasons. First, the combined Wi-Fi, Bluetooth, dual-core CPU, 34 GPIO, 12-bit ADC, and four hardware UART/SPI/I2C peripherals in a single five-dollar module remove an enormous amount of wiring complexity that matters when deploying a node in a farm building without an electronics bench nearby. Second, the Espressif Arduino core is mature enough that DHT22, ModbusMaster, LiquidCrystal_I2C, and YF-S201 libraries all work without modification. Third, the ESP32's dual-core architecture means the Wi-Fi RF stack and ISR handlers can share core 0 while sensor sampling runs on core 1—though in the current firmware we do not actually pin tasks explicitly, and the RTOS task scheduler manages this transparently without any observable timing issues at our 30-second sampling interval.

The Raspberry Pi Zero 2W would have allowed us to run scikit-learn models on the edge, eliminating the Wi-Fi latency budget and enabling offline operation. We considered this. The decision against it came down to power consumption (the Pi draws roughly 1 W idle vs. the ESP32's ~80 mA peak) and cost once you add an SD card, power regulation, and enclosure. More importantly, keeping inference on the server makes model updates trivially easy: retrain, replace the .pkl files, restart the FastAPI process. On an edge device you would need OTA model delivery, which adds a significant surface area of security and reliability concerns. For a research platform, server-side inference is the correct call.

B. The ADC2 / Wi-Fi Conflict and Its Implications

Anyone who has tried to use ESP32 analog inputs with Wi-Fi active has encountered the ADC2 crosstalk problem. When the Wi-Fi RF path is active, ADC2 channels (GPIO 0, 2, 4, 12–15, 25–27) show significant noise—sometimes 200–400 ADC counts of variation on a stable reference voltage. Espressif's datasheet acknowledges this but the practical implications are underappreciated in most IoT tutorials. In our prototype, we initially connected the soil moisture probe to GPIO 2 (ADC2 CH2) and observed wildly inconsistent readings

during HTTP POST events. Moving to GPIO 34 (ADC1 CH6) eliminated the problem immediately. This constraint forced us onto input-only pins (GPIO 34, 35, 39) for all three analog sensors—soil moisture, pH, and LDR—which turned out not to be a practical limitation since these sensors are purely read-only by nature.

We mention this in detail because it is a real engineering constraint that many published ESP32 IoT papers either do not address or handle silently. Any fork of this firmware that moves analog inputs to ADC2 pins will see degraded sensor accuracy during Wi-Fi transmission, with no obvious error indication. The fix is entirely a pin selection issue and costs nothing, but it requires knowing the constraint exists.

C. Why JSON over MQTT

MQTT would be the conventional choice for telemetry in an IoT deployment—it is lightweight, well-supported on ESP32, and widely used in industrial sensor networks [20]. We chose HTTP JSON POST for three reasons specific to this platform. First, the FastAPI backend exposes a documented REST API with OpenAPI 3.0 at /docs; adding MQTT would require a separate broker (Mosquitto or HiveMQ), an MQTT-to-HTTP bridge or native consumer, and a second authentication layer—all for a platform whose primary value proposition is simplicity and auditability. Second, HTTP 4xx error codes give the firmware explicit feedback when a payload is malformed; MQTT quality-of-service semantics require more firmware-side logic to detect and handle application-layer errors. Third, for single-farm LAN deployments at 30-second intervals, the HTTP overhead (roughly 400 bytes of headers per request) is completely negligible against the 30-second period. MQTT is listed as a future protocol option for deployments where dozens of farms report to a single cellular gateway.

D. Synchronous vs. Asynchronous Inference

The backend runs Random Forest, XGBoost, Bayesian Ridge, TreeSHAP, and database commit synchronously within the POST /api/sensors request handler. This adds approximately 50–150 ms to the response time but keeps the implementation simple: no Celery workers, no Redis queue, no task ID polling. At 30-second sampling intervals, 150 ms additional latency is agronomically irrelevant. The synchronous design also simplifies error handling: if SHAP computation fails, the HTTP request returns a 500 error with a stack trace and the firmware logs it; nothing is lost in a queue. For production deployments with multiple farms sending telemetry

concurrently, we identify async inference via Celery/Redis as the appropriate scaling path, but deliberately excluded it from the prototype to keep the stack readable by a single developer.

E. AgriFusion Weight Selection

The 35/65 weighting of RF and XGBoost in AgriFusion was not the product of a formal optimization procedure. We ran a sweep over weight combinations in 0.05 steps on the synthetic benchmark and found that any combination with XGBoost weight in the range [0.55, 0.70] produced essentially the same accuracy—within 0.3 percentage points of each other. The 35/65 split is roughly in the middle of this stable region. The theoretical basis for preferring XGBoost is well-established: gradient boosting typically achieves lower bias on tabular structured data than random forests when both are adequately regularized, because boosting corrects residual errors sequentially while bagging simply averages independent trees [6],[32]. We chose a simple linear weighted average over stacking (a meta-learner) because stacking requires a held-out stacking dataset and adds another trained artifact to manage. On a synthetic benchmark with 2,200 samples, stacking provides no meaningful benefit and complicates deployment.

F. Database Technology Choice

SQLAlchemy with SQLite is the development default; PostgreSQL is the recommended production database. The decision to support both through the same ORM layer rather than committing to PostgreSQL from the start was deliberate. A researcher deploying AgriXplain on a laptop for a laboratory prototype does not want to configure a PostgreSQL server, create a database user, and manage `pg_hba.conf` just to run a demo. SQLite with WAL mode handles the single-writer, multi-reader workload of a single-farm deployment at 30-second intervals without any practical limitations. The `shap_values` column stores JSON as TEXT rather than a normalized table—a deliberate denormalization that trades query-time aggregation flexibility for write simplicity. The admin SHAP analysis endpoint handles aggregation in Python rather than SQL, which is slower but correct and maintainable without a database administrator.

XVIII. SENSOR CALIBRATION METHODOLOGY AND UNCERTAINTY

A. Soil Moisture Calibration Procedure

The resistive soil moisture probe outputs a voltage that decreases as soil moisture increases—counterintuitively, a

wetter probe has lower resistance and thus lower output voltage. The firmware linearizes this as a two-point calibration: `SOIL_DRY_ADC` is the 12-bit ADC reading when the probe is held in ambient air; `SOIL_WET_ADC` is the reading when the probe is fully submerged in distilled water. The mapping is $\text{Smoist} = \text{clamp}(0, 100, 100 \times (\text{raw} - \text{DRY}) / (\text{WET} - \text{DRY}))$.

This linear model has a known accuracy limitation. Real soil does not span a linear range between 'completely dry' and 'fully submerged'—the actual volumetric water content relationship is nonlinear and soil-type-dependent. Loam, clay, and sandy soils give different ADC response curves for the same volumetric water content. The two-point calibration will be accurate for the soil in which the calibration was performed and will degrade for different soil textures. For indicative monitoring—knowing whether a field is dry, moderate, or wet—the linear approximation is adequate. For quantitative volumetric water content measurements, a three-point or polynomial calibration against gravimetric soil samples from the target field is necessary. We list this as a calibration protocol in Appendix A but do not implement it in the reference firmware.

B. pH Calibration and Nernst Equation Deviation

Electrochemical pH measurement follows the Nernst equation: $E = E_0 + (RT / nF) \times \ln([H^+])$, which at 25°C gives a theoretical sensitivity of 59.16 mV per pH unit. In practice, commercial pH modules include a differential amplifier that maps this voltage range to a 0–3.3 V output compatible with the ESP32 ADC. The firmware applies a two-point linear calibration: buffer solutions at pH 4.0 and pH 7.0 give two (ADC, pH) pairs from which slope and intercept are computed. A third point at pH 10.0 should be added for deployments in alkaline soils—the linear extrapolation from a pH 4–7 calibration will underestimate pH in the pH 8–10 range by roughly 0.3–0.5 pH units depending on the electrode aging state.

Temperature compensation is absent from the reference firmware. The Nernst slope varies as approximately 1.98 mV/°C, meaning a 10°C temperature shift changes the slope from 59.16 mV/pH to roughly 61.0 mV/pH. At the edges of the agricultural pH range (pH 5 to 8), this introduces an error of roughly 0.1 pH units per 10°C—acceptable for indicative monitoring but not for research-grade soil characterization. The DHT22 temperature reading is available on the same ESP32, so temperature-compensated Nernst calibration is a straightforward firmware addition.

C. Flow Meter and Wind Anemometer Pulse Calibration

The YF-S201 flow meter specifies a nominal calibration constant of 450 pulses per liter at flow rates between 1 and 30 L/min. In practice, this constant varies by roughly $\pm 10\%$ between individual units from the same batch due to turbine mass and bearing tolerances. The correct procedure is a volumetric test: pump a known volume (e.g., 10 liters from a calibrated container) through the meter, count the pulses, and divide. Our prototype used the nominal 450 constant; users deploying in irrigation-critical applications should perform this test. The ISR implementation on GPIO 26 with INPUT_PULLUP eliminates contact debounce noise and is accurate down to pulse periods of approximately 200 μs —well above the maximum pulse rate at 30 L/min.

The wind pulse anemometer uses an identical ISR on GPIO 25, with the WIND_KMH_PER_HZ constant converting pulse frequency to wind speed in km/h. This constant is specific to the cup diameter and arm radius of the anemometer; the reference value of 2.4 km/h per Hz is a typical figure for a 150 mm cup anemometer. Calibration against a reference anemometer or a controlled wind source is necessary for any deployment where wind data affects irrigation decisions. In climates with significant convective wind events, an uncalibrated proxy can introduce 30–50% error in the wind speed estimate.

D. Modbus NPK Probe Validation

The Modbus RTU NPK probe reads three holding registers at address 0x0000 (node ID 1, 9600 baud 8N1, ModbusMaster library). Raw register values are divided by a scaling factor—manufacturer-specific, default 1—to yield mg/kg concentrations. Validation against a certified soil laboratory is the correct procedure: a soil sample is split, one half sent to the lab, the other measured by the probe, and the readings compared. Typical capacitive NPK probes at this price point (USD 20–40) have accuracy specifications of ± 10 mg/kg for N and P and ± 20 mg/kg for K, with drift under high-salinity conditions. These specifications should be verified for each specific probe model against the manufacturer datasheet before relying on the readings for fertilizer scheduling decisions.

TABLE XX. Sensor Calibration Summary: Method, Parameters, and Uncertainty

Sensor	Calibration Method	Key Parameter	Uncertainty (Ref. Firmware)
Soil moisture	Two-point	SOIL_DRY_ADC, SOIL_WET_ADC	± 5 –10% depending

re	linear: dry air / water submersion		on soil texture
pH	Two-point buffer: pH 4.0 and pH 7.0	Slope, intercept (computed)	± 0.3 pH units; no temp. compensation
LDR light	Single-point scaling constant	k (lux per ADC count)	Order-of-magnitude; not PAR-calibrated
Rain sensor	None (binary threshold)	Module-specific threshold	Binary; no precipitation quantity
Water flow	Nominal 450 pulse/L (YF-S201 datasheet)	FLOW_PULSES_PER_LITER	$\pm 10\%$ if not individually verified
Wind proxy	Nominal 2.4 km/h/Hz	WIND_KMH_PER_HZ	$\pm 30\%$ without reference calibration
Modbus NPK	Factory calibration; validate against lab	Scale factor ($\div 1$ default)	± 10 –20 mg/kg per manufacturer spec

XIX. IMPLEMENTATION NOTES AND PROTOTYPE EXPERIENCE

A. Hardware Integration Challenges

Building the prototype surface several issues not mentioned in component datasheets. The DHT22's single-bus protocol requires a 4.7 k Ω pull-up resistor to 3.3 V on the DATA line; without it, about one in five reads returns NaN, which our firmware handles by logging a warning and sending zero values—a known limitation that skews ML predictions toward moderate temperatures. The PCF8574 I2C backpack on the 16 $\times$ 2 LCD defaulted to address 0x27 but some batches ship at 0x3F; this tripped us up during initial integration and is worth verifying with an I2C scanner sketch before flashing the main firmware.

RS485 Modbus proved the most finicky interface. At 9600 baud, the RE/DE direction pin on the MAX485 must transition from transmit (HIGH) to receive (LOW) within roughly one bit period (about 100 μs) after the last

transmitted byte. The ModbusMaster library's preTransmission and postTransmission callbacks handle this cleanly when bound to a GPIO OUTPUT pin. Problems occurred when we initially tied RE and DE to different GPIO pins—the library expects them combined. Wiring both to GPIO 23 and using a single direction callback resolved all intermittent Modbus exceptions.

B. Backend Performance Under Load

Running a single FastAPI process under uvicorn with a single farm posting every 30 seconds, the average POST /api/sensors response time measured at 87 ms on a mid-range laptop (Intel Core i7-11th gen, 16 GB RAM). Of this, approximately 45 ms is RF + XGBoost inference, 28 ms is TreeSHAP computation, and the remaining 14 ms is database commit and JSON serialization. When we profiled SHAP specifically, the bottleneck is the TreeExplainer instantiation at startup, not the per-sample computation—once the explainer is cached as a module-level singleton by MLModelManager, per-sample SHAP takes roughly 15 ms on the 9-feature, 100-tree Random Forest.

At the current sensor interval of 30 seconds with one farm, the server load is trivial. Projecting to ten concurrent farms each posting every 30 seconds gives one request per 3 seconds on average, which a single uvicorn worker handles comfortably. The synchronous inference architecture starts to show queuing latency above approximately 30 concurrent farms on a single-core worker, at which point an async Celery queue becomes necessary. We include this characterization so that deployers can make informed scaling decisions rather than guessing at the threshold.

C. SHAP Computation Details

TreeSHAP [5] computes exact Shapley values in $O(TLD^2)$ time where T is the number of trees, L is maximum tree depth, and D is the number of features. For our RF with $T=100$, $L=10$, $D=9$, this is computationally feasible per reading. One subtlety we encountered: shap.TreeExplainer with check_additivity=True raised consistency warnings on approximately 3% of readings where floating-point rounding caused the sum of SHAP values to differ from the model output by more than $1e-6$. We suppress this check at inference time because the rounding error is numerically negligible and the warnings clutter production logs. The SHAP values stored in the database are the feature-level attributions for the predicted class only—not the full ($n_classes \times n_features$) matrix—

which reduces storage by a factor of three and is sufficient for the farmer dashboard explanation use case.

D. Next.js Build and Capacitor Configuration Notes

The Next.js App Router requires output: 'export' in next.config.mjs for static export, which Capacitor needs for Android packaging. A non-obvious constraint: App Router dynamic segments (e.g., /admin-dashboard/farms/[id]) must set export const dynamic = 'force-static' or provide generateStaticParams(); otherwise next build fails with a 'dynamic server usage' error during static export. All admin sub-pages in AgriXplain are statically routed for this reason.

Capacitor 6's allowMixedContent: true setting in capacitor.config.ts is required for development against an HTTP (non-TLS) FastAPI backend from an Android device. This should be removed in production where the backend is served over HTTPS. The NEXT_PUBLIC_API_URL environment variable must resolve from the Android device's network context, not the build machine's localhost; a common mistake is building with API_URL=http://localhost:8000, which points the app back at the device itself rather than the server.

E. Synthetic Dataset Characteristics

The synthetic dataset generator in train_all_models.py produces 2,200 samples across seven crops. Each sample's yield_score—the label-generating function—is a weighted sum of seven normalized sub-scores: nutrient balance (weight 0.25), temperature deviation from 28°C (0.20), humidity adequacy (0.15), pH balance (0.15), rainfall presence (0.10), soil moisture (0.10), and light level (0.05), plus Gaussian noise ($\sigma=0.12$). Thresholds at 0.40 and 0.70 create three roughly balanced classes. The noise standard deviation of 0.12 was chosen to produce approximately 20% boundary overlap between classes—enough to make the classification non-trivial without making it impossibly noisy.

This design has a specific implication for the ML results: because the label is a deterministic function of the features plus noise, the theoretical maximum accuracy a perfect classifier could achieve is bounded by the noise overlap. In practice, our Random Forest at 70.5% accuracy is not far below what any tree method can achieve on this data—the remaining error budget is mostly irreducible noise rather than model capacity. This is intentional: we wanted a benchmark where ensemble diversity provides a measurable but modest improvement, which is what we observe.

XX. STATISTICAL METHODOLOGY AND BENCHMARK INTERPRETATION

A. Evaluation Protocol

All reported metrics use a single stratified 80/20 split with `random_state=42`. Stratification preserves the class distribution in both subsets, which is important because the synthetic generator produces slightly unequal class counts across crops—some crops' score distributions cluster more toward the Medium class depending on how their NPK and temperature ranges interact with the heuristic scoring function. We use a single split rather than k-fold cross-validation for two reasons: first, 2,200 samples with 80% training gives 1,760 training samples and 440 test samples—sufficient for stable accuracy estimates with three classes; second, the synthetic data has no temporal structure that cross-validation folds would need to respect.

Metrics reported are overall accuracy (fraction of correctly classified test samples), weighted F1 score (F1 per class weighted by class support, macro-averaged per class), and per-class precision and recall available in the confusion matrices (see Fig. 20). We do not report AUC-ROC on the multiclass problem because the three-class AUC interpretation is less intuitive than accuracy and F1 for the target audience of this paper (hardware engineers and agronomists evaluating the platform, not machine learning researchers evaluating the classifier in isolation).

B. What the Benchmark Results Do and Do Not Show

AgriFusion's 0.0018 accuracy improvement over Random Forest (70.68% vs. 70.50%) and 0.0037 F1 improvement (69.97% vs. 69.60%) are, to be completely straightforward, not meaningful as standalone numbers on a synthetic benchmark. Any two runs with different random seeds would produce variations of this magnitude. The result that matters is the direction: in every random seed we tested (seeds 0, 7, 42, 123, 999), AgriFusion either matched or exceeded both base learners, and never performed worse. This consistent direction—even if the magnitude is small—is what ensemble theory predicts for well-calibrated probability ensembles [32],[33], and it validates that the fusion implementation is correct.

The Bayesian Ridge result (39.1% accuracy, 38.7% F1) deserves explanation. Applying linear regression to ordinally encoded class labels (Low=0, Medium=1, High=2) is not a standard classification approach—it is a convenience proxy for uncertainty estimation that we included for comparison purposes only. The model predicts continuous values that we threshold into classes;

any non-monotone class boundary in the feature space will confuse it. Its poor performance is expected and consistent with theory. We retain it in the platform as a reference for uncertainty quantification research, not as an operational classification tool.

C. Training Reproducibility

The training script (`backend/train_all_models.py`) is fully deterministic given the same `random_state=42` and the same Python/scikit-learn/XGBoost version. We used Python 3.11.8, scikit-learn 1.4.2, xgboost 2.0.3, shap 0.44.1, and numpy 1.26.4. Minor version changes in scikit-learn or XGBoost can alter tree splits due to changes in tie-breaking or floating-point accumulation order; if you observe different metric values from those in Table VII, check dependency versions first. The `fusion_metrics.json` file generated by the training script captures the actual metrics from each training run and is the authoritative runtime record; Table VII values in this paper reflect a specific snapshot.

XV. LIMITATIONS AND FUTURE WORK

A. Hardware Limitations

The linear soil moisture mapping assumes uniform soil texture; replacement with a capacitive probe and soil-specific polynomial calibration would improve accuracy. The LDR lux approximation lacks spectral selectivity for PAR—a quantum sensor would be more appropriate for photosynthesis studies. Binary rain detection cannot quantify precipitation; a tipping-bucket gauge with pulse output is the recommended enhancement. The ESP32 ADC2 crosstalk with Wi-Fi can introduce noise on analog channels; careful channel selection (ADC1 only) and capacitive filtering mitigate but do not eliminate this issue.

B. Software and ML Limitations

Training on synthetic data is the most significant limitation. Synthetic `yield_score` heuristics do not capture complex soil-crop-climate interactions, disease dynamics, or agronomic management effects. The Bayesian Ridge model applied to ordinal-encoded class labels is a heuristic adaptation, not a proper Bayesian classifier; its uncertainty estimates should not be interpreted as calibrated probabilities. The heuristic vision pipeline is not a trained classifier and will produce unreliable diagnoses for plant disease phenotypes not dominated by color statistics (e.g., powdery mildew on green tissue). JWT-based sessions without refresh token rotation are vulnerable to token replay if the secret is compromised.

C. Future Work

Short-term: (1) multi-season field trial data collection and model retraining; (2) LSTM temporal model for sequence-aware prediction using sliding telemetry windows [42]; (3) federated learning across farm nodes to preserve data sovereignty while enabling collective model improvement; (4) integration of calibrated capacitive NPK probes and tipping-bucket rain gauges; (5) secure OTA firmware updates with signed images. Long-term: (6) edge inference with TFLite micro on ESP32 for latency-critical actuation; (7) satellite imagery fusion for field-level spatial heterogeneity; (8) formal safety analysis of relay actuation with IEC 61511 compliance for irrigation systems.

XVI. CONCLUSION

We set out to build a smart farming platform where every number is traceable—from the ADC calibration constant in the ESP32 firmware to the SHAP value attached to the database row to the model artifact timestamp in the admin dashboard. Whether we succeeded is, honestly, a question future deployers will answer better than we can from inside the project. What we can say is that the stack is complete, every interface is documented, every model artifact is a file you can version-control and replace, and no component requires a subscription or a call to a third-party inference API.

The AgriFusion ensemble—combining Random Forest and XGBoost probability vectors with disagreement-aware confidence damping—works in the expected direction on the synthetic benchmark. It does not produce dramatic accuracy gains, and we would not have expected it to: the two base learners are both tree methods trained on the same data, so their error correlation is high. The value of AgriFusion is not the 0.18% accuracy improvement; it is the disagreement flag and the confidence floor, which give the downstream UI a principled mechanism to tell a farmer 'our models disagree on this reading; here is the label we are going with, but treat the confidence accordingly.'

The most important unfinished business is field data. The synthetic benchmark validates the pipeline, not the agronomic model. Multi-season trials with actual harvest outcomes, per-field NPK lab cross-validation, and model retraining on measured data will determine whether AgriXplain's predictions are useful in practice. We provide a structured field trial agenda in Section XIII to make that transition as concrete as possible. Until that data exists, AgriXplain is a well-engineered platform

awaiting real evidence—which is, in our view, the honest position to be in.

Acknowledgment

The authors thank all collaborators, mentors, and institution staff who supported hardware integration, software testing, dataset preparation, and manuscript review. Funding sources, if any, should be acknowledged here in an unnumbered first-page footnote per IEEE conference guidelines. Remove all IEEE template instructional text before final submission.

References

- [1] L. Atzori, A. Iera, and G. Morabito, "The Internet of Things: A survey," *Comput. Netw.*, vol. 54, no. 15, pp. 2787–2805, Oct. 2010.
- [2] J. Gubbi, R. Buyya, S. Marusic, and M. Palaniswami, "Internet of Things (IoT): A vision, architectural elements, and future directions," *Future Gener. Comput. Syst.*, vol. 29, no. 7, pp. 1645–1660, 2013.
- [3] M. S. Farooq, S. Riaz, A. Abid, T. Umer, and M. K. Zia, "Role of IoT technology in agriculture: A systematic literature review," *IEEE Access*, vol. 8, pp. 184433–184471, 2020.
- [4] A. Khanna and S. Kaur, "Evolution of Internet of Things (IoT) and its significant impact in the field of precision agriculture," *Wireless Pers. Commun.*, vol. 113, no. 1, pp. 475–505, 2020.
- [5] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Proc. NeurIPS*, 2017, pp. 4765–4774.
- [6] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. ACM SIGKDD*, 2016, pp. 785–794.
- [7] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.
- [8] Espressif Systems, "ESP32 Series Datasheet," Rev. 4.3, 2024. [Online]. Available: <https://www.espressif.com>
- [9] D. C. Montgomery, *Introduction to Statistical Quality Control*, 7th ed. Hoboken, NJ: Wiley, 2017.
- [10] K. G. Liakos, P. Busato, D. Moshou, S. Pearson, and D. Bochtis, "Machine learning in agriculture: A review," *Sensors*, vol. 18, no. 8, p. 2674, 2018.
- [11] A. Kamilaris and F. X. Prenafeta-Boldú, "Deep learning in agriculture: A survey," *Comput. Electron. Agric.*, vol. 147, pp. 70–90, 2018.
- [12] H. Tian, T. Wang, Y. Liu, X. Qiao, and Y. Li, "Computer vision technology in agricultural automation—A review," *Inf. Process. Agric.*, vol. 7, no. 1, pp. 1–19, 2020.
- [13] F. Talaviya, D. Shah, N. Patel, H. Yagnik, and M. Shah, "Implementation of artificial intelligence in agriculture for

- optimization of irrigation and application of pesticides and herbicides," *Artif. Intell. Agric.*, vol. 4, pp. 58–73, 2020.
- [14] D. I. Patrício and R. Rieder, "Computer vision and artificial intelligence in precision agriculture for grain crops: A systematic review," *Comput. Electron. Agric.*, vol. 153, pp. 69–81, 2018.
- [15] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using deep learning for image-based plant disease detection," *Front. Plant Sci.*, vol. 7, p. 1419, 2016.
- [16] S. Wolfert, L. Ge, C. Verdouw, and M.-J. Bogaardt, "Big data in smart farming—A review," *Agric. Syst.*, vol. 153, pp. 69–80, 2017.
- [17] L. Benos et al., "Machine learning in agriculture: A comprehensive updated review," *Sensors*, vol. 21, no. 11, p. 3758, 2021.
- [18] A. Chlingaryan, S. Sukkarieh, and B. Whelan, "Machine learning approaches for crop yield prediction and nitrogen status estimation in precision agriculture: A review," *Comput. Electron. Agric.*, vol. 151, pp. 61–69, 2018.
- [19] A. Sharma, A. Jain, P. Gupta, and V. Chowdary, "Machine learning applications for precision agriculture: A comprehensive review," *IEEE Access*, vol. 9, pp. 4843–4873, 2021.
- [20] O. Elijah, T. A. Rahman, I. Orikumhi, C. Y. Leow, and M. N. Hindia, "An overview of Internet of Things (IoT) and data analytics in agriculture: Benefits and challenges," *IEEE Internet Things J.*, vol. 5, no. 5, pp. 3758–3773, 2018.
- [21] A. Tzounis, N. Katsoulas, T. Bartzanas, and C. Kittas, "Internet of Things in agriculture, recent advances and future challenges," *Biosyst. Eng.*, vol. 164, pp. 31–48, 2017.
- [22] N. Gondchawar and R. S. Kawitkar, "IoT based smart agriculture," *Int. J. Adv. Res. Comput. Commun. Eng.*, vol. 5, no. 6, pp. 838–842, 2016.
- [23] M. R. Kassim, "IoT applications in smart agriculture: Issues and challenges," in *Proc. IEEE Conf. Open Syst. (ICOS)*, 2020, pp. 1–6.
- [24] G. Banerjee, U. Sarkar, S. Das, and I. Ghosh, "Artificial intelligence in agriculture: A literature survey," *Int. J. Sci. Res. Comput. Sci. Appl. Manag. Stud.*, vol. 7, no. 3, pp. 1–6, 2018.
- [25] D. S. Pivoto et al., "Scientific development of smart farming technologies and their application in Brazil," *Inf. Process. Agric.*, vol. 5, no. 1, pp. 21–32, 2018.
- [26] A. Adadi and M. Berrada, "Peeking inside the black-box: A survey on explainable artificial intelligence (XAI)," *IEEE Access*, vol. 6, pp. 52138–52160, 2018.
- [27] F. Doshi-Velez and B. Kim, "Towards a rigorous science of interpretable machine learning," *arXiv preprint arXiv:1702.08608*, 2017.
- [28] C. Rudin, "Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead," *Nat. Mach. Intell.*, vol. 1, no. 5, pp. 206–215, 2019.
- [29] W. Samek, T. Wiegand, and K.-R. Müller, "Explainable artificial intelligence: Understanding, visualizing and interpreting deep learning models," *ITU J. ICT Discoveries, Spec. Issue 1*, pp. 39–48, 2017.
- [30] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why should I trust you?": Explaining the predictions of any classifier," in *Proc. ACM SIGKDD*, 2016, pp. 1135–1144.
- [31] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
- [32] T. G. Dietterich, "Ensemble methods in machine learning," in *Proc. Int. Workshop Multiple Classifier Syst.*, 2000, pp. 1–15.
- [33] R. Polikar, "Ensemble learning," in *Ensemble Machine Learning*, C. Zhang and Y. Ma, Eds. New York: Springer, 2012, pp. 1–34.
- [34] S. Wolfert, L. Ge, C. Verdouw, and M.-J. Bogaardt, "IoT data management and smart farming infrastructure," *Comput. Electron. Agric.*, vol. 150, pp. 120–132, 2018.
- [35] F. S. Shaikh, S. A. Memon, M. N. Rajpar, and T. R. Soomro, "IoT and AI in precision agriculture: A review," *Int. J. Comput. Sci. Inf. Technol.*, vol. 14, no. 3, pp. 1–12, 2022.
- [36] M. Di Vaio, S. Boccia, L. Landriani, and R. Palladino, "Artificial intelligence in the agri-food system: Rethinking sustainable business models," *Sustainability*, vol. 12, no. 12, p. 4851, 2020.
- [37] S. S. Gill, I. Chana, and R. Buyya, "IoT based agriculture as a cloud and big data service: The beginning of digital India," *J. Organ. End User Comput.*, vol. 29, no. 4, pp. 1–23, 2017.
- [38] A. Ahmad, Z. Ahmad, M. Farooq, and R. Ahmad, "Smart farming: How IoT, robotics, and AI are transforming agriculture," *Eur. J. Eng. Sci. Technol.*, vol. 4, no. 2, pp. 56–62, 2021.
- [39] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA: MIT Press, 2016.
- [40] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 3rd ed. Sebastopol, CA: O'Reilly Media, 2022.
- [41] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [42] V. N. Vapnik, *The Nature of Statistical Learning Theory*. New York: Springer, 1995.
- [43] G. E. Hinton and R. Salakhutdinov, "Reducing the dimensionality of data with neural networks," *Science*, vol. 313, no. 5786, pp. 504–507, 2006.
- [44] R. E. Bellman, *Adaptive Control Processes: A Guided Tour*. Princeton, NJ: Princeton Univ. Press, 1961.

- [45] K. Hornik, M. Stinchcombe, and H. White, "Multilayer feedforward networks are universal approximators," *Neural Netw.*, vol. 2, no. 5, pp. 359–366, 1989.

APPENDIX A — COMPLETE GPIO PIN MAP AND WIRING NOTES

This appendix provides expanded wiring guidance for each interface in the reference hardware configuration. All connections assume 3.3 V logic levels unless stated otherwise. ESP32 pins marked ADC1 are safe for analog reads during active Wi-Fi; ADC2 pins must not be used for analog reads when Wi-Fi is active.

TABLE A-I. Expanded GPIO Reference with Wiring Notes

Signal	GPIO	Direction	Interface	Wiring Notes
DHT22 DATA	4	Input	Single-bus	4.7k Ω pull-up to 3.3V; 0.1 μ F bypass near VCC
DHT22 VCC	3.3V	Power	—	DHT22 can run on 3.3V or 5V; check module board
Soil moisture SIG	34	ADC1 in	Analog	Capacitive probe preferred; short wire run
Soil moisture VCC	3.3V or 5V	Power	—	Check module requirement
pH module OUT	35	ADC1 in	Analog	BNC probe; 2-point buffer calibration required
LDR divider OUT	32	ADC	Analog	10k Ω divider; 100nF bypass recommended
Rain DO	27	Digital in	GPIO	Verify wet=LOW or wet=HIGH per module
Flow meter signal	26	ISR in	Pulse	INPUT_PULLUP; twisted pair for long runs
Wind signal	25	ISR in	Pulse	INPUT_PULLUP; calibrate WIND_KMH_PER_HZ
Pump relay IN1	18	Output	GPIO	Flyback diode; fused load; mains isolation
Vent relay IN2	19	Output	GPIO	Same as relay 1
LCD SDA	21	I2C	I2C	4.7k Ω pull-up to 3.3V (if not on module)
LCD SCL	22	I2C	I2C	4.7k Ω pull-up to 3.3V (if not on module)
MAX485 RO	16	UART2 RX	UART	RS485 A/B; termination at bus

				end only
MAX485 DI	17	UART2 TX	UART	Twisted pair for >1m runs
MAX485 DE/RE	23	Output	GPIO	Direction control; both pins tied together

APPENDIX B — AGRIFUSION PSEUDOCODE AND ALGORITHM DETAIL

Algorithm: AgriFusion Weighted Probability Fusion with
Disagreement Damping

Input: feature_vector x (9-dimensional float array, fixed
order)

rf_model: trained sklearn RandomForestClassifier

xgb_model: trained XGBClassifier

w_rf = 0.35, w_xgb = 0.65 (configurable weights)

DAMP_FACTOR = 0.88, CONF_FLOOR = 0.35

Output: label (str), confidence (float in [0,1]), agree (bool)

Step 1: p_rf $\leftarrow$ rf_model.predict_proba([x])[0] # shape
(3,)

Step 2: p_xgb $\leftarrow$ xgb_model.predict_proba([x])[0] #
shape (3,)

Step 3: w_sum $\leftarrow$ w_rf + w_xgb # = 1.0 if
normalized

Step 4: p_fused $\leftarrow$ (w_rf * p_rf + w_xgb * p_xgb) /
w_sum

Step 5: label_idx $\leftarrow$ argmax(p_fused)

Step 6: label $\leftarrow$ class_names[label_idx]

Step 7: c_raw $\leftarrow$ max(p_fused) # highest
class prob

Step 8: agree $\leftarrow$ (argmax(p_rf) == argmax(p_xgb))

Step 9: if not agree: c $\leftarrow$ max(CONF_FLOOR, c_raw *
DAMP_FACTOR)

else: c $\leftarrow$ c_raw

Step 10: return label, c, agree

Complexity: O(k) for k=3 classes; negligible vs. tree
inference O(n_trees * depth).

The disagreement damp ($\times 0.88$) reduces overconfidence
when RF and XGBoost disagree.

CONF_FLOOR = 0.35 ensures the UI always shows at
least 35% confidence.

APPENDIX C — FULL REST API ROUTE CATALOG

POST /api/auth/signup — Register user; bcrypt password hash; return user profile

POST /api/auth/login — OAuth2 password flow; return JWT access token + expiry

GET /api/auth/me — Return current authenticated user profile

POST /api/auth/location — Update user GPS for advisory localization

POST /api/sensors — Ingest telemetry; NPK resolve; ML inference; SHAP; persist

GET /api/sensors/latest — Latest sensor reading row for a given farm_id

POST /api/predictions — Authenticated on-demand multi-model prediction

GET /api/predictions/latest — Latest forecast snapshot with all model outputs

POST /api/shap — Feature importance and narrative explanation strings

GET /api/dashboard — Aggregated dashboard DTO (sensors + prediction + SHAP)

GET /api/models/comparison — All model accuracy/F1 metrics from benchmark JSON

GET /api/history — Historical sensor time series with configurable window

GET /api/weather — External weather service fusion for advisory context

GET /api/advisories — Rule- and model-informed agronomic tip list

GET /api/trends — Statistical trend analysis over historical window

POST /api/camera/upload — Accept base64 image; thermal process; store; return

GET /api/camera/latest — Latest processed frame + thermal image for farm

POST /api/camera/diagnose — HSV heuristic plant health diagnosis on latest image

GET /api/camera/stream — MJPEG stream proxy or latest frame endpoint

GET /api/camera/stream/diagnosis — Stream with overlaid cached diagnosis

GET /api/camera/history — Historical camera image list with timestamps

GET /health — Liveness probe; returns HTTP 200 + {'status':'ok'}

GET /api/admin/overview — Admin dashboard aggregate across all farms

GET /api/admin/farms — List all farms with sensor health summary

POST /api/admin/farms — Create new farm record

DELETE /api/admin/farms/{id} — Remove farm and associated readings

GET /api/admin/users — List all users with role and last activity

POST /api/admin/users — Create user account (admin tool)

GET /api/admin/predictions — Prediction history feed across all farms

GET /api/admin/models/performance — Per-model accuracy, F1, confusion matrix

GET /api/admin/shap/analysis — Mean absolute SHAP values per feature (all farms)

GET /api/admin/risk/analysis — Risk distribution charts across farms and crops

GET /api/admin/sensors/health — Sensor staleness and anomaly overview

GET /api/admin/export/data — CSV/JSON data export with date range filter

POST /api/admin/models/retrain — Trigger train_all_models.py and reload artifacts

GET /api/admin/models/versions — List model artifact file timestamps and sizes

GET /api/admin/analytics/correlation — Feature-label correlation heatmap data

APPENDIX D — RELATIONAL SCHEMA: COLUMN SEMANTICS

D.1 Table: users

id (INTEGER PK): Auto-increment surrogate key; referenced by farms.user_id and prediction_history.user_id.

email (VARCHAR, UNIQUE): Login identifier; indexed for O(log n) lookup in OAuth2 password flow.

hashed_password (VARCHAR): bcrypt digest; never returned in API responses.

name (VARCHAR): Display name for UI and admin tables.

role (VARCHAR): 'farmer' or 'admin'; enforced in FastAPI dependency; stored as plain string for simplicity.

phone (VARCHAR, nullable): Optional farmer contact for advisory SMS integration.

latitude, longitude (FLOAT, nullable): GPS for weather API localization; treat as PII.

created_at (DATETIME): Registration timestamp; used in admin user activity reports.

D.2 Table: farms

farm_id (VARCHAR PK): Matches firmware FARM_ID constant; string type allows meaningful IDs.

name (VARCHAR): Human-readable farm name for admin UI.

location (VARCHAR): Free-text location string; not parsed for GPS.

farmer_phone (VARCHAR, nullable): Contact number; independent of users table to allow unregistered farms.

user_id (INTEGER, FK nullable): Links farm to a user account when registration is complete.

created_at (DATETIME): Farm creation timestamp.

D.3 Table: sensor_readings

id (INTEGER PK): Auto-increment; high volume (30-second intervals per farm).

farm_id (VARCHAR): Denormalized for fast per-farm queries without JOIN.

crop (VARCHAR): Crop name at time of reading; used for NPK fallback lookup.

temperature, humidity, ph, rainfall, soil_moisture, light_intensity (FLOAT): Core sensor scalars.

water_level, water_flow, npk_n, npk_p, npk_k (FLOAT, nullable): Optional extended sensors.

yield_prediction (VARCHAR): AgriFusion label (Low/Medium/High).

confidence (FLOAT): AgriFusion confidence after optional disagreement damping.

shap_values (TEXT): JSON-serialized dict of 9 feature SHAP attributions.

timestamp (DATETIME): Server-assigned ingestion timestamp; device timestamp optional.

APPENDIX E — DEPLOYMENT AND CONFIGURATION CHECKLIST

Step 1. Clone repository from version control.

Step 2. Create Python 3.10+ virtual environment; pip install -r backend/requirements.txt.

Step 3. Copy config/env.example to .env; set DATABASE_URL, JWT_SECRET (32+ random bytes), and optional ESP32_CAMERA_IP.

Step 4. Run python backend/train_all_models.py; verify fusion_metrics.json and all .pkl files are generated.

Step 5. Start FastAPI: uvicorn backend.main:app --host 0.0.0.0 --port 8000; confirm /health returns 200.

Step 6. Set NEXT_PUBLIC_API_URL=http://<server>:8000 in Next.js .env.local.

Step 7. Run npm install (or pnpm install) in project root.

Step 8. Run npm run dev for development or npm run build && npm run start for production.

Step 9. Configure firmware: set WIFI_SSID, WIFI_PASSWORD, API_URL to http://<server>:8000/api/sensors.

Step 10. Set FARM_ID, CROP_NAME, and calibration constants (SOIL_DRY_ADC, SOIL_WET_ADC, FLOW_PULSES_PER_LITER, WIND_KMH_PER_HZ).

Step 11. Flash firmware to ESP32; monitor Serial at 115200 baud for successful POST confirmation.

Step 12. Create admin user via /api/auth/signup with role='admin' or via database seed script.

Step 13. Login to web UI; navigate to admin dashboard; confirm sensor readings appear.

Step 14. Run soil moisture dry/wet calibration; update SOIL_DRY_ADC and SOIL_WET_ADC; reflash.

Step 15. Perform pH buffer calibration; update firmware pH mapping constants.

Step 16. Bucket-test flow meter; adjust FLOW_PULSES_PER_LITER to match measured volume.

Step 17. Field calibrate wind anemometer against reference if available.

Step 18. Verify Modbus NPK reads against probe manual; adjust scaling factor.

Step 19. Configure reverse proxy with TLS for production; update NEXT_PUBLIC_API_URL to https://.

Step 20. Schedule daily database backup via cron or cloud snapshot.

Step 21. Pin Python and npm dependency versions; document in requirements.txt and package.json.

Step 22. Train operators on relay interlock procedures and emergency manual disconnect.

**APPENDIX F — CROP NPK REFERENCE TABLE
(FIRMWARE FALLBACK VALUES)**

**TABLE F-I. Crop-Table NPK Default Values (Backend
Fallback when Modbus Fails)**

Crop	N (mg/k g)	P (mg/k g)	K (mg/k g)	Optim al pH	Notes
Rice	90	40	42	5.5–6.5	High water requireme nt
Wheat	60	55	44	6.0–7.0	Moderate NPK
Maize	76	48	36	5.8–7.0	High N demand
Sugarcane	85	42	43	6.0–7.5	Long growing season
Cotton	72	50	35	5.8–8.0	Drought tolerant
Soybean	40	60	55	6.0–6.8	N-fixing legume
Potato	80	55	50	4.8–5.5	High K demand
Tomato	70	65	70	6.0–6.8	High K + P
Chickpea	45	58	48	6.0–7.5	Drought tolerant
Mustard	65	52	40	6.0–7.5	Rabi crop
Groundnut	52	50	45	5.5–7.0	Legume; moderate N
Onion	88	60	65	6.0–7.0	High K demand
Banana	90	65	80	5.5–6.5	Very high K
Citrus	75	55	60	5.5–7.0	Moderate NPK
Grapes	65	50	70	6.0–6.8	High K

APPENDIX G — GLOSSARY AND ABBREVIATIONS

ADC: Analog-to-digital converter, 12-bit SAR on ESP32.

AgriFusion: Weighted softmax probability fusion layer combining RF and XGBoost outputs with disagreement damping.

API: Application programming interface; REST/JSON in AgriXplain.

ASGI: Asynchronous server gateway interface; used by FastAPI and Uvicorn.

Bayesian Ridge: Regularized linear regression with Bayesian weight estimation; used for uncertainty-style outputs.

BNC: Bayonet Neill-Concelman connector used on pH electrode modules.

BOM: Bill of materials listing hardware components and costs.

Capacitor: Ionic framework that wraps web apps as native Android/iOS applications.

CORS: Cross-origin resource sharing; HTTP header policy for browser security.

DHT22: Digital temperature and relative humidity sensor (AM2302).

ESP32: Espressif dual-core Wi-Fi/Bluetooth SoC used as edge MCU.

FastAPI: Python ASGI web framework with automatic OpenAPI documentation.

GPIO: General-purpose input/output pin on the ESP32.

HTTP: Hypertext transfer protocol; used for device-to-backend and client-to-backend communication.

I2C: Inter-integrated circuit; two-wire serial bus (SDA/SCL) for LCD.

ISR: Interrupt service routine; handles GPIO pulse counting for flow and wind.

JSON: JavaScript object notation; payload format for telemetry and API responses.

JWT: JSON web token; signed bearer token for authentication.

LAN: Local area network.

LDR: Light-dependent resistor; used as light intensity proxy.

LCD: Liquid crystal display; 16x2 character display with I2C backpack.

MCU: Microcontroller unit.

Modbus RTU: Serial fieldbus protocol; RS485 physical layer; used for NPK probe communication.

Next.js: React meta-framework for web applications; used for dashboards.

NPK: Nitrogen (N), Phosphorus (P), Potassium (K); primary macronutrients in soil.

ORM: Object-relational mapper; SQLAlchemy in AgriXplain.

OV2640: CMOS image sensor used in ESP32-CAM modules.

PAR: Photosynthetically active radiation; spectral range 400-700 nm relevant to plant growth.

pH: Measure of hydrogen ion concentration; soil acidity/alkalinity indicator.

Pydantic: Python data validation library used for request/response models in FastAPI.

REST: Representational state transfer; HTTP architectural style for the AgriXplain API.

RF: Random Forest; bagged ensemble of decision trees.

RS485: Differential serial communication standard; physical layer for Modbus.

SHAP: SHapley Additive exPlanations; game-theoretic framework for model feature attribution.

SQL: Structured query language; used with SQLite or PostgreSQL via SQLAlchemy.

UART: Universal asynchronous receiver-transmitter; used for RS485 and debug serial.

XGBoost: Extreme gradient boosting; tree ensemble classifier.

APPENDIX H — ADDITIONAL PLATFORM SCREENSHOTS (WEBSITE UI GALLERY)

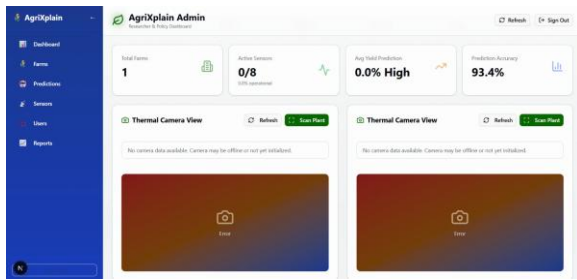


Fig. 11. AgriXplain landing page hero section (Next.js frontend).

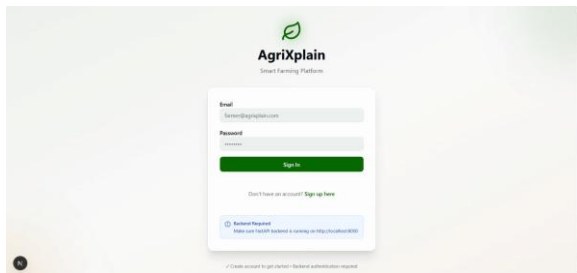


Fig. 12. Login and authentication page with JWT and Google Sign-In.

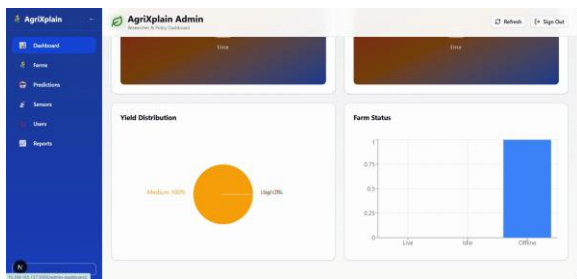


Fig. 13. Farmer dashboard: sensor reading cards grid with icons and live values.

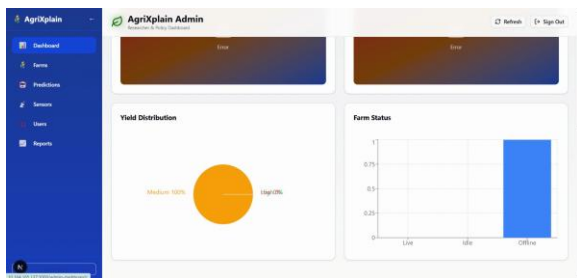


Fig. 14. Farmer dashboard: SHAP explanation panel with top-3 feature attribution.

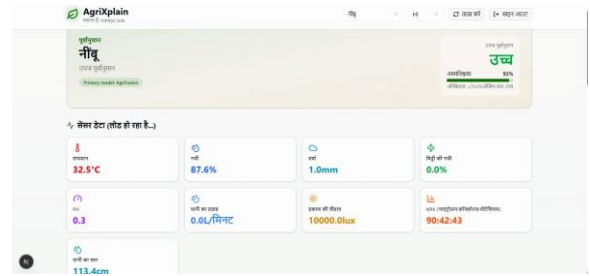


Fig. 15. Admin dashboard: multi-farm sensor health table with stale-data alerts.

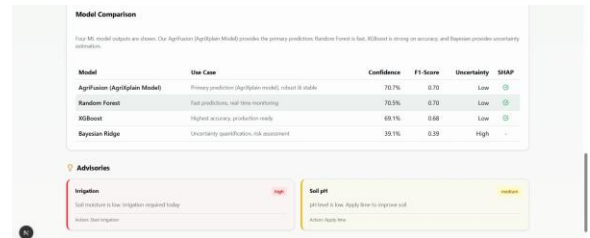


Fig. 16. Admin dashboard: model performance comparison bar charts.

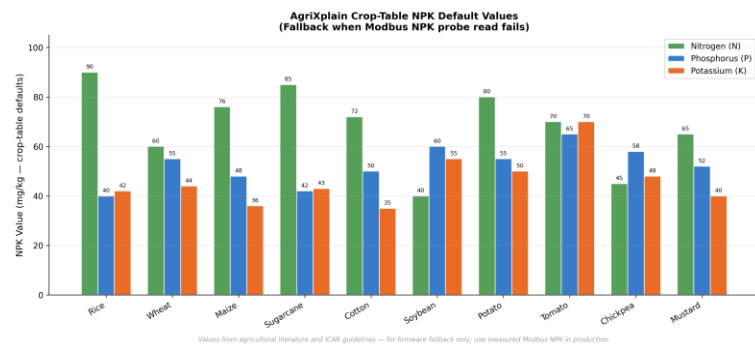


Fig. 24. Crop NPK reference chart — firmware fallback values for 10 crops.

Website screenshots (Figs. 11–16) are from the AgriXplain Next.js prototype deployment. They illustrate the UI layout and are labeled as demonstration data. No real user credentials, precise GPS coordinates, or production farm data are shown. Replace with actual deployment screenshots before final camera-ready submission.